

Laporan Tugas Besar 1
IF3270 - Pembelajaran Mesin
Feedforward Neural Network



Disusun oleh:

Kelompok : matthew dont cry

Nazwan Siddqi Muttaqin/18223066

Surya Suharna/18223075

Matthew Sebastian Kurniawan/18223096

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

I. Daftar Isi

I. Daftar Isi.....	2
II. Deskripsi Persoalan.....	3
III. Pembahasan.....	4
A. Penjelasan Implementasi.....	4
B. Splitting Data.....	11
C. Hasil Pengujian.....	11
IV. Kesimpulan dan Saran.....	48
A. Kesimpulan.....	48
B. Saran.....	49
V. Pembagian Tugas.....	50
VI. Referensi.....	52
VII. Lampiran.....	53

II. Deskripsi Persoalan

Tugas Besar I mata kuliah IF3270 Pembelajaran Mesin bertujuan agar mahasiswa dapat mengimplementasikan algoritma Feedforward Neural Network (FFNN) secara mandiri dari awal (from scratch). Model FFNN yang dibangun harus bersifat dinamis, yaitu mampu menerima konfigurasi jumlah layer dan neuron yang bervariasi. Selain itu, arsitektur jaringan harus mendukung berbagai fungsi aktivasi (Linear, ReLU, Sigmoid, tanh, Softmax), perhitungan loss (MSE, Binary Cross-Entropy, Categorical Cross-Entropy), serta metode inisialisasi bobot (Zero, Uniform, Normal).

Proses pelatihan model mewajibkan adanya implementasi forward propagation dan backward propagation dengan menerapkan aturan rantai (chain rule) yang dapat memproses data dalam bentuk batch. Pembaruan bobot model dilakukan menggunakan algoritma Gradient Descent. Sistem juga dituntut untuk memiliki fitur regularisasi L1 dan L2.

Tahap akhir dari tugas ini adalah pengujian model menggunakan dataset Global Student Placement & Salary untuk melakukan klasifikasi pada fitur prediksi `placement_status`. Pengujian mencakup eksperimen dan analisis mendalam terkait pengaruh modifikasi hyperparameter (seperti `depth`, `width`, fungsi aktivasi, dan `learning rate`), penerapan regularisasi, serta perbandingan performa akurasi dengan model library `sklearn MLP`.

III. Pembahasan

A. Penjelasan Implementasi

1. Deskripsi Kelas

Kelas FFNN

Lokasi: src/model.py

Peran	Atribut	Method
Merepresentasikan keseluruhan jaringan FFNN.	layer_sizes: arsitektur jaringan, misalnya [input, hidden1, ..., output].	init(...): validasi konfigurasi, inisialisasi semua layer, aktivasi, loss, normalisasi.
Mengelola inisialisasi layer, forward, backward, update bobot, training loop, plotting, dan persistence.	activation_names: nama aktivasi per layer transisi.	_progress_bar(current, total, width=28): utilitas progress bar.
	loss_name: nama loss yang dipakai.	_regularization_loss(l1_lambda, l2_lambda): hitung penalti regularisasi.
	_act, _loss_obj, _init: instance helper untuk aktivasi, loss, dan inisialisasi.	forward(x): forward pass batch input.
	_act_fns, _act_derivs: daftar fungsi aktivasi dan turunannya untuk tiap layer.	backward(y_true, y_pred): backprop batch untuk mendukung jalur khusus softmax+CCE.
	_loss_fn, _loss_deriv: fungsi loss aktif dan turunannya.	update_weights(learning_rate, l1_lambda, l2_lambda): gradient descent + regularisasi + update gamma RMSNorm.
	_use_combined_softmax_cce: flag optimasi gradien gabungan softmax + CCE.	train(...): training per epoch/batch, support verbose 0/1, return history.
	layers: list DenseLayer.	predict(x): inferensi.

	norms: list RMSNorm atau None per layer	save(filename) dan load(filename): simpan/muat model via pickle.
	_net_cache, _act_cache, _norm_cache: cache intermediate untuk backprop.	plot_loss(): grafik train/val loss.
	history: menyimpan train_loss dan val_loss per epoch.	plot_weight_distribution(layer_indices): histogram bobot layer tertentu.
		plot_gradient_distribution(layer_indices): histogram gradien bobot layer tertentu.

Kelas DenseLayer

Lokasi: src/layer.py

Peran	Atribut	Method
Menyediakan operasi affine transform untuk satu layer dense.	weights, biases: parameter yang dilatih.	init(n_inputs, n_neurons, init_method, **kwargs): inisialisasi bobot (zero/uniform/normal) dan bias.
	dweights, dbiases: gradien parameter.	forward(inputs): hitung $net = XW + b$.
	inputs: cache input untuk backward.	backward(d_net): hitung dweights, dbiases, dinputs.
	dinputs: gradien terhadap input layer.	

Kelas ActivationFunctions

Lokasi: src/activation_function.py

Peran	Atribut	Method
Menyediakan fungsi aktivasi yang dapat dipilih per layer, termasuk	Tidak menyimpan state permanen (stateless utility class).	linear, linear_derivative
		relu, relu_derivative

turunan pertama.		sigmoid, sigmoid_derivative
		tanh, tanh_derivative
		softmax, softmax_derivative
		Bonus: elu, elu_derivative, swish, swish_derivative

Kelas LossFunctions

Lokasi: src/loss_function.py

Peran	Atribut	Method
Menyediakan fungsi objektif pelatihan dan turunannya terhadap prediksi.	Tidak menyimpan state permanen (stateless utility class).	mean_squared_error, mean_squared_error_derivative
		binary_cross_entropy, binary_cross_entropy_derivative
		categorical_cross_entropy, categorical_cross_entropy_derivative

Kelas Initialization

Lokasi: src/initialization.py

Peran	Atribut	Method
Utility untuk inisialisasi parameter bobot sesuai metode yang dipilih.	Tidak menyimpan state, tiap method menggunakan RandomState berbasis seed.	zero(shape)
		uniform(shape, lower_bound, upper_bound, seed)
		normal(shape, mean, variance, seed)
		Bonus: xavier(shape, distribution, seed)
		Bonus: he(shape, distribution, seed)

Kelas RMSNorm

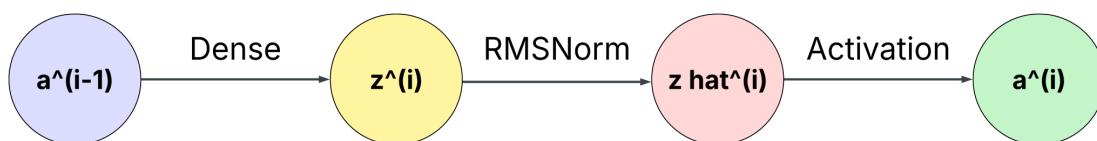
Lokasi: src/rmsnorm.py

Peran	Atribut	Method
Normalisasi aktivasi linear per layer sebelum aktivasi non-linear (opsional).	eps: konstanta stabilitas.	init(dim, eps)
	dim: dimensi fitur.	forward(x): hitung RMS norm dan scaling gamma.
	gamma: parameter skala yang dilatih.	backward(grad_output): gradien terhadap input dan gamma.
	dgamma: gradien gamma.	update(learning_rate): update parameter gamma.
	cache_input, cache_rms: cache untuk backward.	

Jadi jika dirangkum lebih singkat relasi antar kelasnya adalah sebagai berikut

No.	Relasi
1.	FFNN menggunakan DenseLayer sebagai blok layer.
2.	FFNN memanggil ActivationFunctions dan LossFunctions secara dinamis via nama method.
3.	FFNN menggunakan Initialization untuk set awal weights.
4.	FFNN opsional menyisipkan RMSNorm di tiap layer transisi.
5.	ffnn_model hanya re-export agar import FFNN lebih sederhana.

2. Forward Propagation



a. Fase Inisialisasi dan Input

Sebelum data mengalir, model melakukan validasi dan penyiapan cache untuk kebutuhan Backpropagation nantinya.

- i. Input Shape:
Data x masuk dengan dimensi (N, D), di mana N adalah batch size dan D adalah jumlah fitur.
- ii. Cache:
Model mengosongkan list self._net_cache (nilai pre-aktivasi) dan self._norm_cache (nilai setelah RMSNorm).
- iii. Activation Cache: self._act_cache diinisialisasi dengan menyertakan input mentah x sebagai elemen pertama ($a^{(0)}$) .

Kenapa? Karena gradien layer pertama membutuhkan input asli untuk menghitung turunan terhadap bobot (dW).

- b. Loop Transformasi Layer ($i = 1 \dots L$)
Model melakukan iterasi sebanyak jumlah layer yang didefinisikan dalam self.layers.
 - i. Transformasi Linear (Dense Layer)
Data dari layer sebelumnya ($a^{(i-1)}$) dikalikan dengan matriks bobot dan ditambah bias menggunakan operasi layer.foward(input).

$$z^{(i)} = a^{(i-1)}W^{(i)} + b^{(i)}$$

- ii. Normalisasi (Optional RMSNorm)
Jika pada layer i diaktifkan RMSNorm, maka nilai $z^{(i)}$ akan dinormalisasi untuk menjaga stabilitas distribusi aktivasi.
 - a. Menghitung nilai root mean square

$$rms = \sqrt{\frac{1}{d} \sum z^2 + \epsilon}$$

- b. Normalize & Scale

$$\hat{z}^{(i)} = \frac{z^{(i)}}{rms} \cdot \gamma$$

Nilai tersebut kemudian disimpan ke _norm_cache.

c. Transformasi Non-Linear (Activation)

Nilai pre-aktivasi (baik $z^{(i)}$ atau $z \text{ hat}^{(i)}$) dilewatkan ke fungsi aktivasi yang dipilih (ReLU, Sigmoid, dan lainnya).

$$a^{(i)} = \sigma(z^{(i)})$$

Hasil tersebut akan disimpan ke `self.act_cache` dan menjadi input untuk layer $i+1$.

d. Output & Prediksi Akhir

Setelah loop selesai, elemen terakhir dari `self._act_cache` adalah hasil prediksi model ($y \text{ hat}$).

- i. Binary Classification: Menggunakan sigmoid di layer terakhir untuk menghasilkan probabilitas $[0, 1]$.
- ii. Multi-class Classification: Menggunakan softmax untuk menghasilkan distribusi probabilitas yang totalnya adalah 1.
- iii. Regression: Biasanya menggunakan aktivasi linear.

3. Backward Propagation & Weight Update

a. Inisialisasi Gradien

Proses dimulai di `src/model.py` tepat setelah forward pass selesai. Model menentukan titik awal gradien berdasarkan fungsi loss yang digunakan.

i. Jalur Optimasi (Softmax + CCE)

Jika model menggunakan kombinasi ini, gradien dihitung secara efisien untuk menghindari ketidakstabilan numerik.

$$\frac{\partial L}{\partial z_{out}} = \frac{y \text{ hat} - y}{N}$$

Di mana N adalah jumlah sampel dalam batch

ii. Jalur umum (loss lain)

Model memanggil turunan fungsi loss di `src/loss_function.py`.

$$\delta_{out} = \text{loss_derivative}(y, y \text{ hat})$$

b. Propagasi Chain Rule

Model melakukan iterasi mundur (reversed) melalui list layer. Setiap tahapan melibatkan koordinasi antara `model.py`, `activation_function.py`, `rmsnorm.py`, dan [layer.py](#).

i. Turunan Aktivasi

Gradien dari layer setelahnya dikalikan dengan turunan fungsi aktivasi layer saat ini.

$$\delta^{(l)} = \delta^{(l+1)} \circ f'(z^{(l)})$$

Catatan Khusus Softmax: Jika tidak menggunakan jalur optimasi, model melakukan Vector-Jacobian Product untuk menangani dependensi antar-kelas pada output softmax.

ii. RMSNorm (Opsional)

Jika layer memiliki modul normalisasi, gradien dilewatkan ke `RMSNorm.backward()` di `src/rmsnorm.py`. Di sini, gradien terhadap parameter skala (γ) dan gradien terhadap input dihitung.

iii. Komputasi Gradien pada Dense Layer

1. Gradien Bobot (dW): Menentukan seberapa besar perubahan bobot memengaruhi error.

$$\frac{\partial L}{\partial W^{(l)}} = (A^{(l-1)})^T \cdot \delta^{(l)}$$

2. Gradien Bias (db): Akumulasi error per neuron dalam satu batch.

$$\frac{\partial L}{\partial b^{(l)}} = \sum \delta^{(l)}$$

3. Gradien Input (dA): Sinyal error yang diteruskan ke layer sebelumnya ($l-1$).

$$\frac{\partial L}{\partial A^{(l-1)}} = \delta^{(l)} \cdot (W^{(l)})^T$$

c. Weight Update

Setelah semua gradien terkumpul di setiap objek `DenseLayer`, model melakukan langkah perbaikan.

i. Regularisasi (Weight Decay)

Sebelum update, gradien bobot dimodifikasi dengan penalti untuk mencegah *overfitting*.

1. L1 (Lasso) dengan menambahkan λ dikalikan $\text{sign}(W)$ untuk mendorong bobot menjadi nol/sparse.

2. L2 (Ridge) dengan menambahkan lambda 2 dikalikan dengan W untuk mencegah bobot menjadi terlalu besar.
- ii. Eksekusi Gradient Descent
Setiap parameter diperbarui dengan arah berlawanan dari gradien, dikalikan dengan *learning rate*.

$$W = W - \eta(dW + \text{reg gradien})$$

$$b = b - \eta.db$$

Jika menggunakan RMSNorm, parameter γ juga diperbarui dengan cara yang sama.

B. Splitting Data

Dalam tahap pengujian, keseluruhan dataset yang berjumlah 10.000 sampel dibagi secara acak menjadi tiga bagian, yaitu data latih (*train set*) sebesar 70%, data validasi (*validation set*) sebesar 15%, dan data uji (*test set*) sebesar 15%. Pembagian ini kami lakukan karena setiap bagian memiliki peran yang saling melengkapi untuk menghasilkan evaluasi yang objektif. Data latih bertugas untuk memperbarui bobot model selama proses pembelajaran, sementara data validasi digunakan di tengah pelatihan untuk memantau performa, menyesuaikan hyperparameter, dan mencegah model sekadar menghafal data (*overfitting*). Dengan mengisolasi data uji hingga akhir pelatihan, kami memastikan bahwa model dievaluasi secara realistis menggunakan data yang benar-benar baru, sehingga hasil pengujian tidak bias dan dapat mencerminkan performa model yang sebenarnya.

C. Hasil Pengujian

1. Pengaruh Depth dan Width

- A) Pengaruh Depth

Metode	Train Acc	Val Acc
Depth = 1	0.7684	0.7473
Depth = 3	0.7867	0.7387
Depth = 5	0.8114	0.7460

- a) Depth = 1: Menghasilkan akurasi training terendah (76.84%), namun memberikan akurasi validasi tertinggi (74.73%) serta gap (jarak) terkecil antara Train, Val, dan Test. Ini menunjukkan struktur yang lebih dangkal sangat efektif menjaga kemampuan generalisasi model dan mencegah overfitting.
- b) Depth = 3: Memberikan peningkatan pada akurasi training (78.67%), namun akurasi validasinya justru menurun (73.87%). Jaringan mulai menunjukkan kecenderungan menghafal data latih dibandingkan memahami pola umumnya.
- c) Depth = 5: Mencapai akurasi training tertinggi secara signifikan (81.14%). Meskipun begitu, model ini menghasilkan gap yang sangat lebar antara akurasi Train dan Val/Test. Kompleksitas model yang terlalu tinggi membuatnya terjebak menghafal detail spesifik data latih (overfitting).

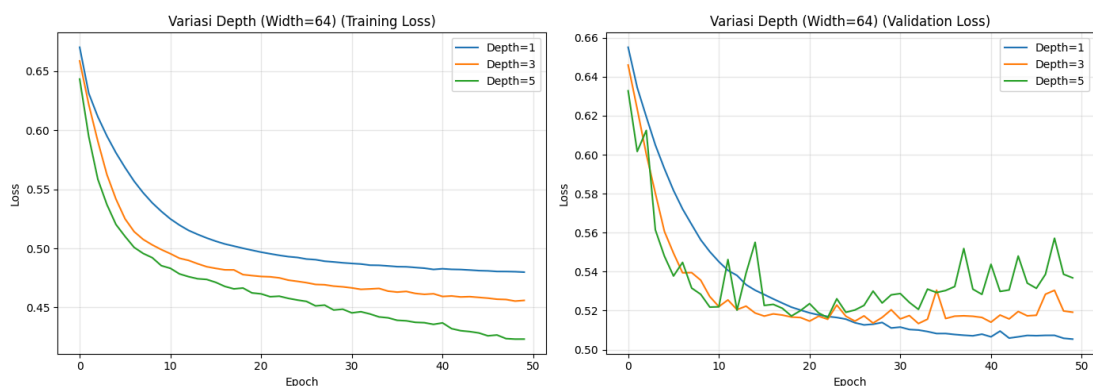
Kurva Loss

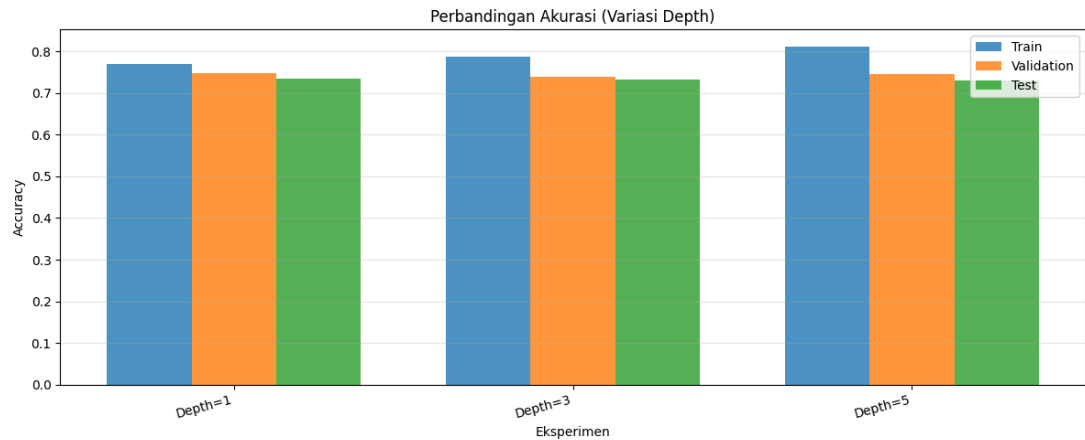
1. Training loss

Model dengan kedalaman tertinggi atau Depth=5 (hijau) mencapai loss terendah paling cepat dan penurunannya sangat tajam. Ini wajar karena model dengan parameter yang sangat banyak memiliki kapasitas besar dan bebas melakukan "*memorizing*" pada data latih.

2. Validation loss

Kurva Depth=1 (biru) terlihat paling halus, stabil, dan terus menurun hingga akhir epoch. Sebaliknya, kurva Depth=5 (hijau) menunjukkan fluktuasi yang sangat tajam dan mulai menunjukkan tren naik di pertengahan hingga akhir epoch (gejala kuat overfitting). Kurva Depth=3 (oranye) berada di tengahnya dengan sedikit fluktuasi.





B) Pengaruh Width

Metode	Train Acc	Val Acc
Width = 16	0.7651	0.7253
Width = 64	0.7780	0.7400
Width = 128	0.7959	0.7413

- Width = 16: Memiliki akurasi training terendah (76.51%) dan akurasi validasi terendah (72.53%) dibandingkan konfigurasi lainnya. Namun, gap antara performa pada data latih dan uji tidak terlalu besar, mengindikasikan bahwa meskipun kapasitas model terbatas dalam mempelajari fitur, model ini cukup stabil dan tidak terlalu overfitting.
- Width = 64: Memberikan keseimbangan yang cukup baik. Akurasi training meningkat menjadi 77.80% dan akurasi validasinya cukup tinggi (74.00%). Penambahan jumlah neuron memberikan kapasitas yang lebih besar bagi model untuk merepresentasikan pola kompleks tanpa kehilangan kemampuan generalisasi secara drastis.
- Width = 128: Menghasilkan akurasi training tertinggi (79.59%), namun akurasi validasinya (74.13%) hanya sedikit lebih baik dibandingkan Width=64. Lebarnya gap antara akurasi training dan test mengindikasikan bahwa penambahan unit yang terlalu banyak pada satu lapisan (dengan Depth=3) mulai mendorong model menuju overfitting, di mana model terlalu fokus menghafal spesifikasi data latih.

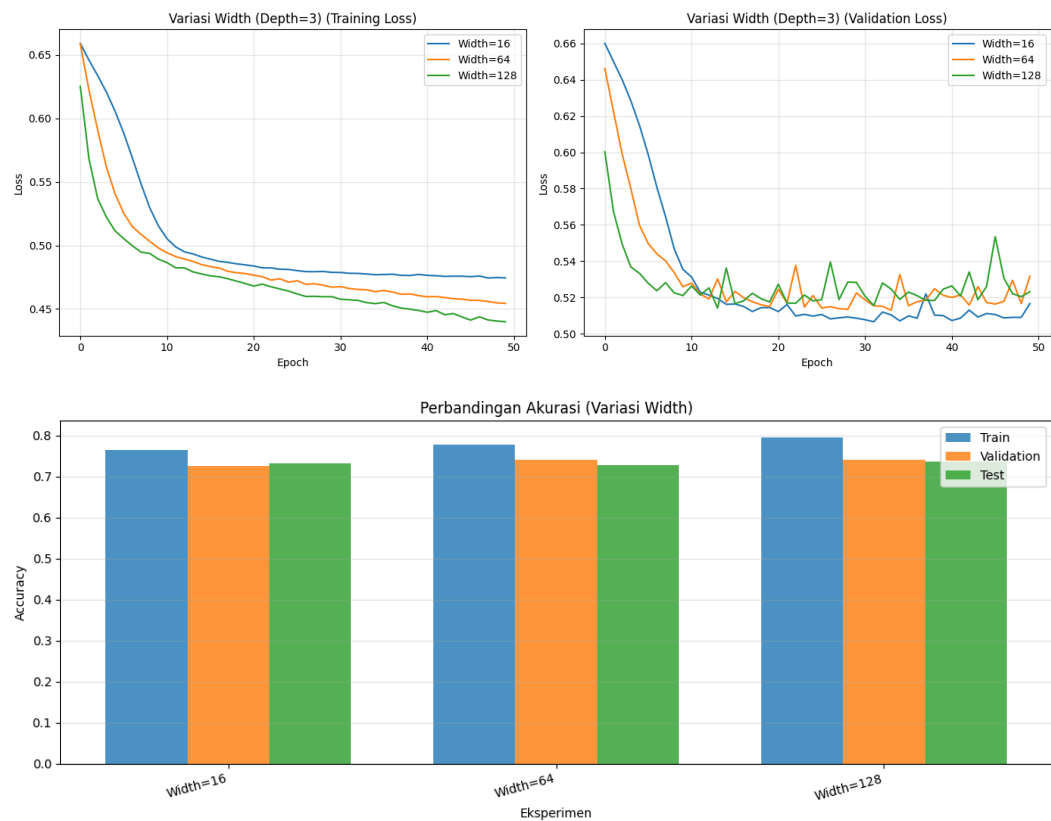
Kurva Loss

1. Training loss

Kurva dengan Width=128 (hijau) mengalami penurunan loss yang paling cepat dan mencapai titik terendah. Hal ini logis mengingat model dengan jumlah unit terbanyak memiliki fleksibilitas tertinggi untuk meminimalkan kesalahan pada data latih secara langsung.

2. Validation loss

Pada kurva Validation Loss, model Width=16 (biru) terlihat paling mulus dan secara konsisten menurun hingga titik konvergensinya di akhir epoch, menandakan pembelajaran yang paling stabil. Sebaliknya, kurva Width=128 (hijau) dan Width=64 (oranye) mulai memperlihatkan fluktuasi (noise) yang cukup terlihat setelah epoch ke-10, yang merupakan indikasi awal bahwa kapasitas model mulai berlebihan untuk pola generalisasi dari dataset tersebut.



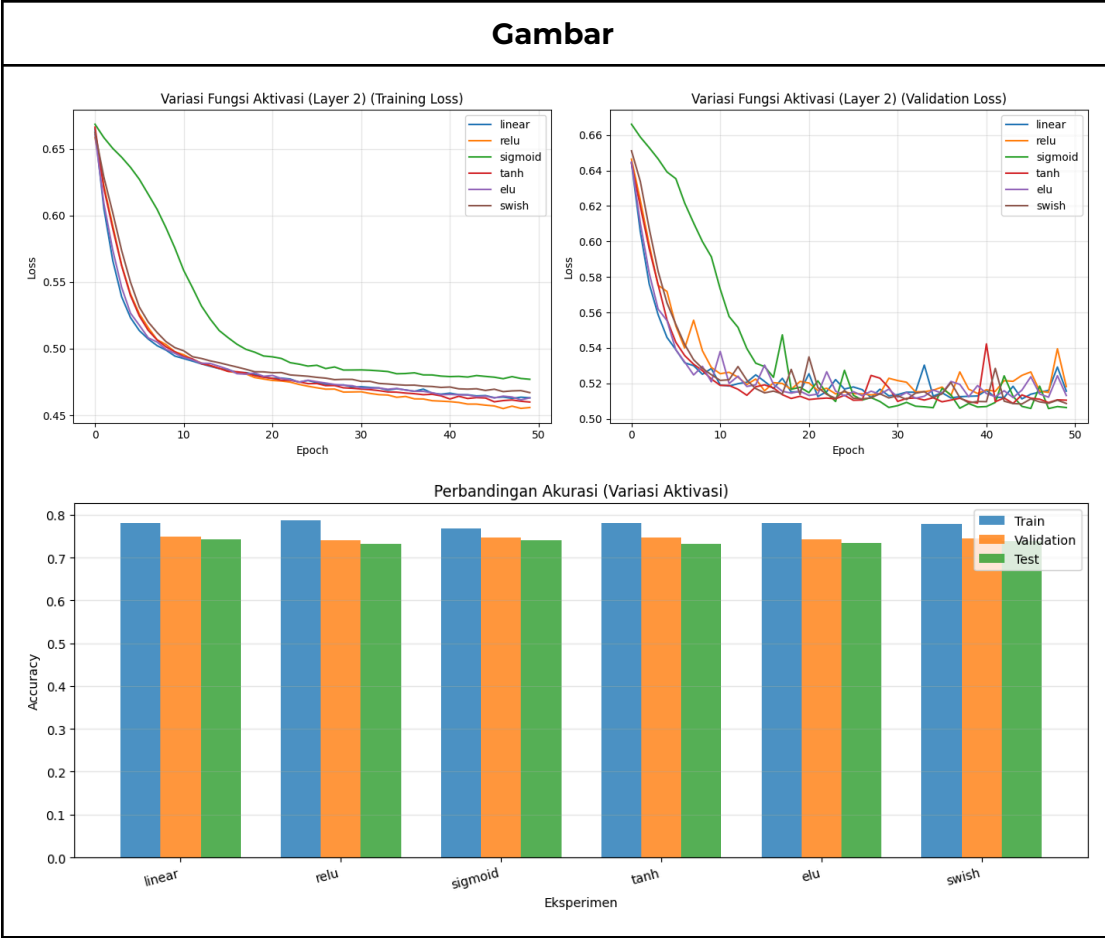
2. Pengaruh Fungsi Aktivasi

Metode	Train Acc	Val Acc
Linear	0.7801	0.7480
ReLU	0.7870	0.7393
Sigmoid	0.7680	0.7467
Tanh	0.7803	0.7460
ELU (Bonus)	0.7813	0.7420
Swish (Bonus)	0.7793	0.7453

- Linear: Menghasilkan akurasi validasi tertinggi (74.80%) dengan gap terkecil terhadap training (78.01%). Ini menunjukkan transformasi linear sangat efektif menjaga kemampuan generalisasi model tanpa menambah kompleksitas yang tidak perlu.
- ReLU: Menghasilkan akurasi training tertinggi (78.70%) namun dengan akurasi validasi terendah (73.93%). Ini menunjukkan ReLU sangat agresif mempelajari data latih, namun rentan memicu overfitting.
- Sigmoid: Memberikan akurasi training terendah (76.80%) dengan akurasi validasi yang cukup baik (74.67%). Ini menunjukkan model belajar terlalu lambat akibat sifat kurvanya yang rentan memicu vanishing gradient.
- Tanh: Memberikan performa yang seimbang dengan akurasi training 78.03% dan validasi 74.60%. Ini menunjukkan sifatnya yang zero-centered efektif mempercepat konvergensi dibandingkan dengan Sigmoid.
- ELU: Mencapai akurasi training tinggi (78.13%) dengan akurasi validasi 74.20%. Ini menunjukkan ELU efektif mengatasi masalah dying neuron yang ada pada ReLU, meskipun gap akurasinya sedikit lebih lebar.
- Swish: Menghasilkan performa yang stabil dengan akurasi training 77.93% dan validasi 74.53%. Ini menunjukkan kurvanya yang halus efektif menjaga aliran gradien tetap baik tanpa memicu overfitting yang terlalu dini.

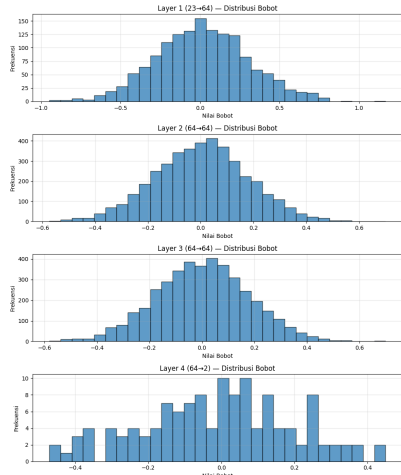
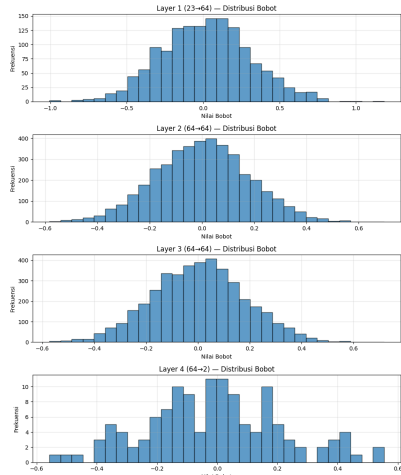
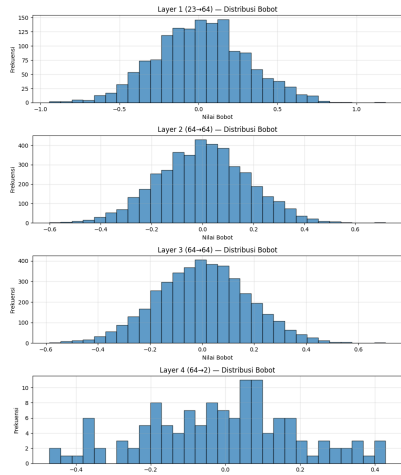
Kurva Loss

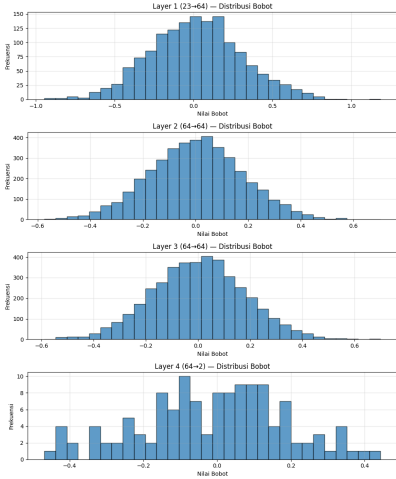
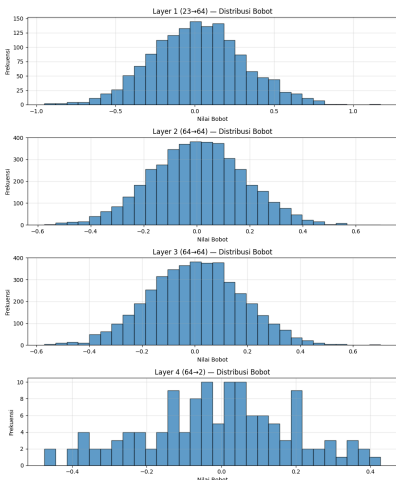
Metode	Penjelasan
Linear	Menghasilkan akurasi validasi tertinggi (74.80%). Kurva training dan validation loss menurun paling mulus dan konstan hingga akhir epoch tanpa fluktuasi, menunjukkan stabilitas tinggi dan terhindar dari overfitting.
ReLU	Mencapai akurasi training tertinggi (78.70%) namun validasi terendah (73.93%). Meski training loss turun tajam dan tercepat, validation loss-nya sangat fluktuatif dan mulai menanjak di pertengahan epoch, mengindikasikan overfitting yang kuat.
Sigmoid	Memberikan akurasi training terendah (76.80%). Penurunan training loss terlihat paling lambat akibat risiko vanishing gradient, namun tren validation loss-nya sangat stabil dan terus menurun beriringan tanpa lonjakan berarti.
Tanh	Mencapai akurasi training 78.03%. Training loss konvergen jauh lebih cepat dari Sigmoid berkat sifatnya yang zero-centered, namun validation loss mulai menunjukkan sedikit fluktuasi (noise) di epoch-epoch akhir.
ELU (Bonus)	Menghasilkan akurasi training tinggi (78.13%) dengan konvergensi loss yang cepat. Namun, gap yang melebar dan munculnya lonjakan kecil pada validation loss di akhir epoch menandakan model mulai rentan menghafal data latih.
Swish (Bonus)	Memberikan performa seimbang (akurasi validasi 74.53%). Penurunan training dan validation loss bergerak cukup selaras. Fluktuasi di akhir epoch jauh lebih terkendali dibanding ReLU, menunjukkan keseimbangan yang solid antara adaptasi dan generalisasi.



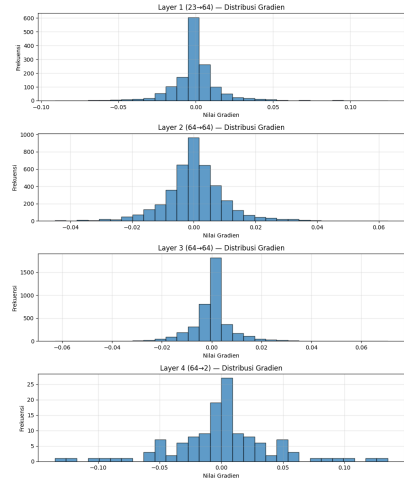
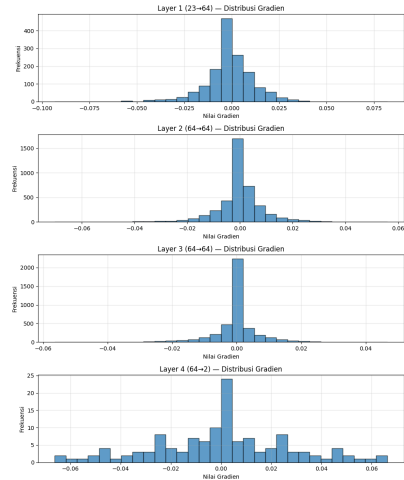
Distribusi Bobot

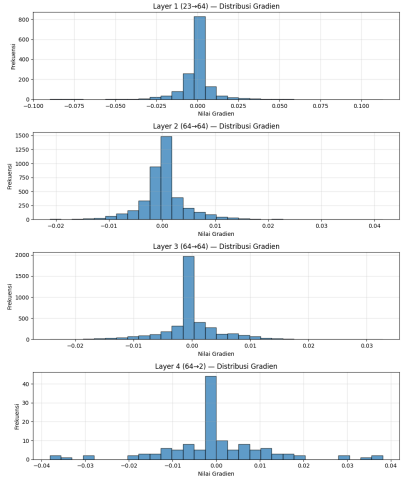
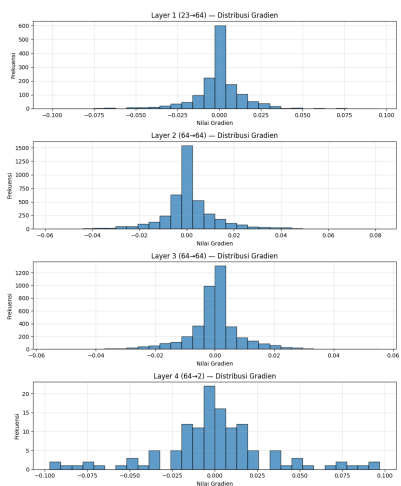
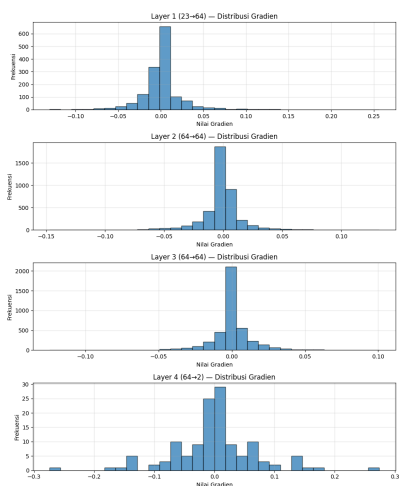
Metode	Gambar	Penjelasan
Linear	The figure shows four histograms of weight distributions for different layers: Layer 1 (23-64) – Distribusi Bobot: A normal distribution centered at 0, with a range from -1.0 to 1.0. Layer 2 (64-64) – Distribusi Bobot: A normal distribution centered at 0, with a range from -0.6 to 0.6. Layer 3 (64-64) – Distribusi Bobot: A normal distribution centered at 0, with a range from -0.6 to 0.6. Layer 4 (64-2) – Distribusi Bobot: A skewed, asymmetric distribution centered around 0, with a range from -0.4 to 0.4.	Layer 1-3 berdistribusi normal (terpusat di 0) dengan rentang yang semakin menyempit, menunjukkan aliran sinyal yang stabil. Sebaliknya, distribusi Layer 4 (output) tampak acak dan asimetris karena bobot harus menyesuaikan diri secara spesifik untuk memisahkan kelas target.

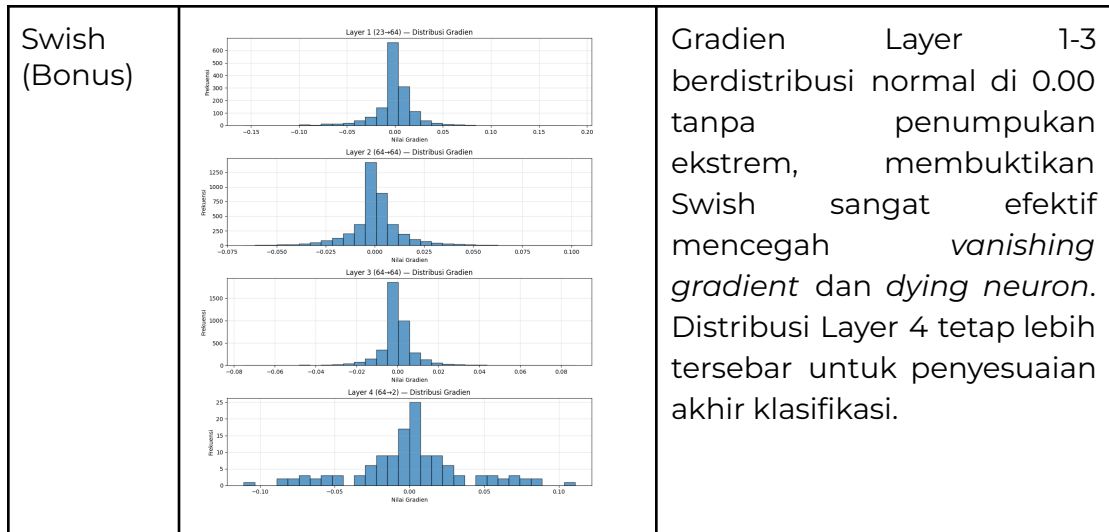
ReLU	 Layer 1 (23-64) – Distribusi Bobot Layer 2 (64-64) – Distribusi Bobot Layer 3 (64-64) – Distribusi Bobot Layer 4 (64-2) – Distribusi Bobot	Mirip dengan Linear, Layer 1-3 membentuk distribusi normal (terpusat di 0) dengan rentang yang semakin menyempit, menandakan aliran fitur antar lapisan yang stabil. Layer 4 (<i>output</i>) terdistribusi secara acak dan tidak beraturan karena bobot beradaptasi kuat secara spesifik untuk memisahkan kelas target akhir.
Sigmoid	 Layer 1 (23-64) – Distribusi Bobot Layer 2 (64-64) – Distribusi Bobot Layer 3 (64-64) – Distribusi Bobot Layer 4 (64-2) – Distribusi Bobot	Konsisten dengan fungsi lainnya, Layer 1-3 mempertahankan distribusi normal (terpusat di 0) dengan rentang yang menyempit setelah lapisan pertama, menjaga stabilitas perambatan sinyal. Layer 4 (<i>output</i>) tetap terdistribusi secara acak dan tersebar sebagai bentuk adaptasi spesifik untuk memisahkan kelas target.
Tanh	 Layer 1 (23-64) – Distribusi Bobot Layer 2 (64-64) – Distribusi Bobot Layer 3 (64-64) – Distribusi Bobot Layer 4 (64-2) – Distribusi Bobot	Serupa dengan fungsi sebelumnya, Layer 1-3 menampilkan distribusi normal (terpusat di 0) dengan rentang penyebaran yang menyempit di lapisan yang lebih dalam, menandakan aliran fitur yang stabil. Layer 4 (<i>output</i>) tetap terdistribusi secara acak dan asimetris sebagai bentuk penyesuaian akhir untuk memisahkan kelas target.

ELU (Bonus)	 Layer 1 (23-64) – Distribusi Bobot Layer 2 (64-64) – Distribusi Bobot Layer 3 (64-64) – Distribusi Bobot Layer 4 (64-2) – Distribusi Bobot	Konsisten dengan pola sebelumnya, Layer 1-3 membentuk distribusi normal (terpusat di 0) dengan rentang yang menyempit pada lapisan tengah, menjaga stabilitas perambatan sinyal. Layer 4 (<i>output</i>) tetap terdistribusi secara acak dan asimetris sebagai penyesuaian akhir untuk memisahkan kelas target secara spesifik.
Swish (Bonus)	 Layer 1 (23-64) – Distribusi Bobot Layer 2 (64-64) – Distribusi Bobot Layer 3 (64-64) – Distribusi Bobot Layer 4 (64-2) – Distribusi Bobot	Seperti fungsi lainnya, Layer 1-3 mempertahankan distribusi normal (terpusat di 0) dengan rentang yang menyempit di lapisan tengah, menunjukkan stabilitas perambatan fitur yang sangat baik. Layer 4 (<i>output</i>) tetap terdistribusi secara acak dan asimetris sebagai bentuk penyesuaian akhir untuk memisahkan kelas target.

Distribusi Gradien

Metode	Gambar	Penjelasan
Linear		Seluruh <i>layer</i> menunjukkan gradien yang terpusat sangat rapat di angka 0.00, menandakan model telah mencapai konvergensi (pembaruan bobot sudah minimal). Persebaran yang sedikit lebih lebar pada Layer 4 (<i>output</i>) menunjukkan masih adanya penyesuaian minor tahap akhir untuk meminimalkan <i>error</i>.
ReLU		Terjadi lonjakan frekuensi yang sangat ekstrem di angka 0.00 pada Layer 1-3. Ini adalah indikasi kuat fenomena <i>dying ReLU</i>, di mana banyak neuron "mati" dan berhenti memperbarui bobotnya. Distribusi Layer 4 tetap lebih tersebar sebagai penyesuaian akhir kelas target.

Sigmoid		Menumpuknya gradien di angka 0.00 pada Layer 1-3 adalah bukti kuat <i>vanishing gradient</i>, di mana pembaruan bobot terhambat dan model belajar sangat lambat. Persebaran Layer 4 tetap sedikit lebih luas untuk penyesuaian akhir klasifikasi.
Tanh		Gradien Layer 1-3 berdistribusi normal di sekitar 0.00 tanpa penumpukan tajam, membuktikan Tanh sangat efektif mencegah <i>vanishing gradient</i>. Distribusi Layer 4 tetap lebih luas untuk penyesuaian akhir klasifikasi.
ELU (Bonus)		Gradien Layer 1-3 terpusat di 0.00 tanpa penumpukan ekstrem seperti ReLU, membuktikan efektivitas ELU dalam mencegah masalah <i>dying neuron</i> dan menjaga aliran gradien tetap stabil. Distribusi Layer 4 tetap lebih luas untuk penyesuaian akhir klasifikasi.



3. Pengaruh Learning Rate

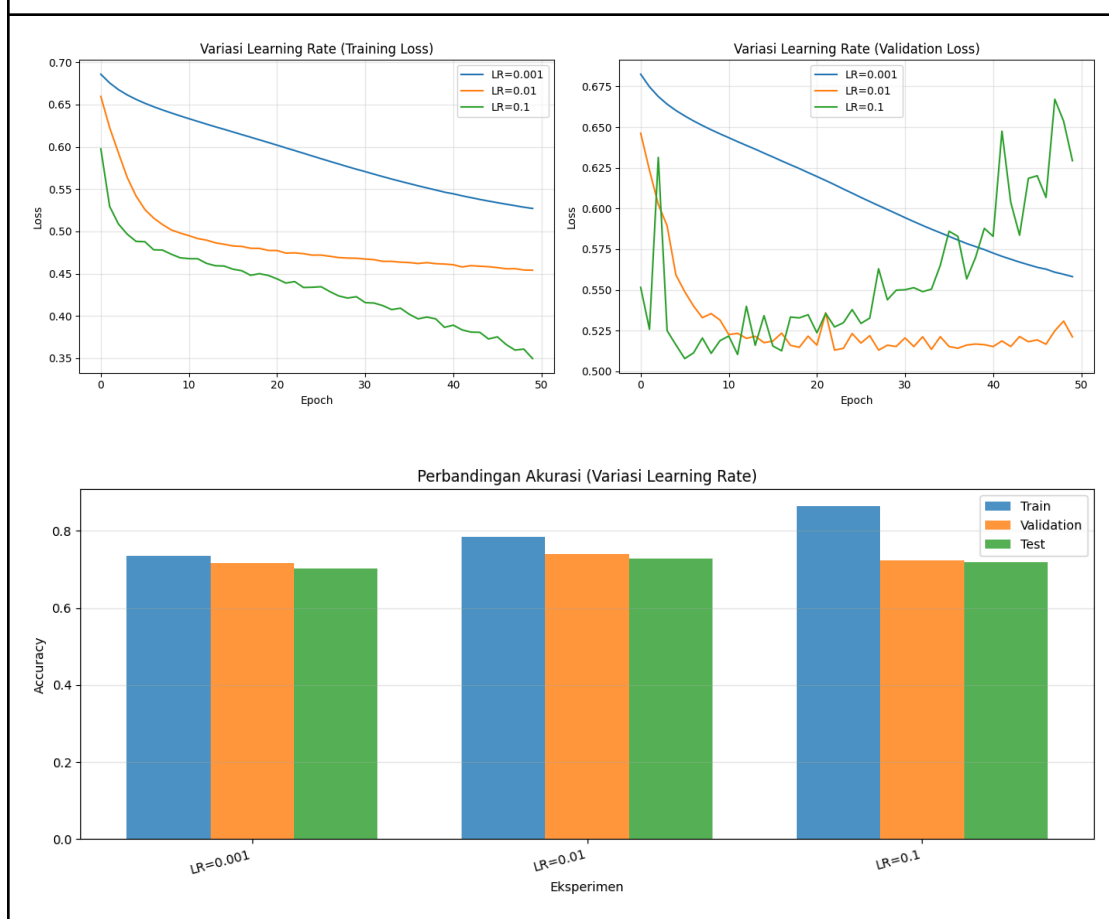
Metode	Train Acc	Val Acc
LR=0.001	0.7361	0.7167
LR=0.01	0.7839	0.7407
LR=0.1	0.8650	0.7227

- LR=0.001: Menghasilkan akurasi training terendah (73.61%) dan validasi terendah (71.67%) dengan gap yang sempit. Ini menunjukkan model belajar terlalu lambat dan belum mencapai konvergensi maksimalnya.
- LR=0.01: Memberikan akurasi validasi tertinggi (74.07%) dengan akurasi training yang seimbang (78.39%). Ini menunjukkan laju pembelajaran yang paling optimal untuk mengadaptasi data tanpa merusak kemampuan generalisasi model.
- LR=0.1: Mencapai akurasi training tertinggi (86.50%) namun akurasi validasinya justru menurun (72.27%). Gap yang sangat lebar ini merupakan indikasi kuat terjadinya overfitting karena langkah pembaruan bobot yang terlalu agresif.

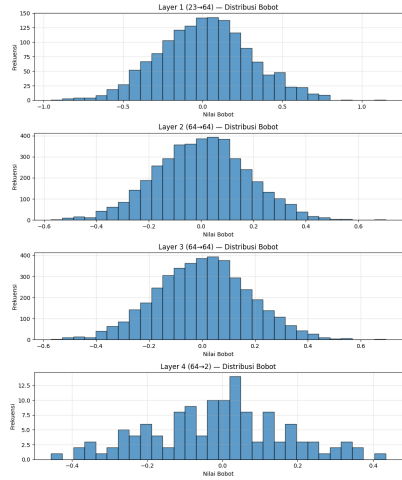
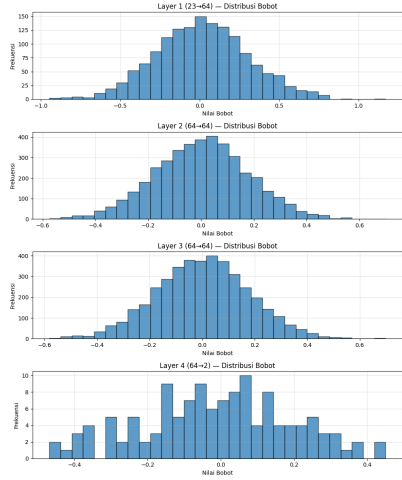
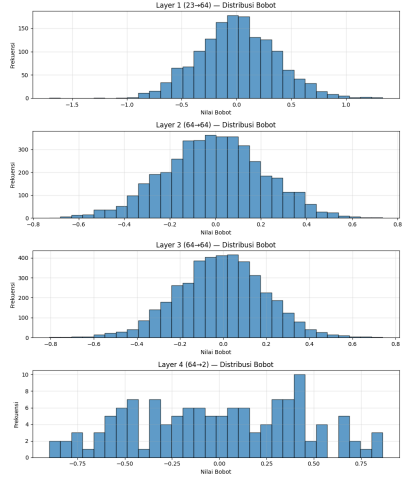
Metode	Penjelasan
LR=0.001	Penurunan kurva <i>training</i> dan <i>validation loss</i> sangat lambat dan konstan. Model belum mencapai konvergensi hingga akhir <i>epoch</i> , menunjukkan laju pembelajaran terlalu kecil (lambat belajar).

LR=0.01	<i>Training</i> dan <i>validation loss</i> turun dengan cepat dan stabil. Kurva validasi berhasil konvergen di titik terendah dengan fluktuasi minor, membuktikan ini adalah laju pembelajaran yang paling optimal.
LR=0.1	Meski <i>training loss</i> turun paling drastis, kurva <i>validation loss</i> sangat fluktuatif dan menanjak tajam setelah pertengahan <i>epoch</i> . Ini adalah bukti kuat <i>overfitting</i> karena langkah pembaruan bobot yang terlalu besar dan tidak stabil.

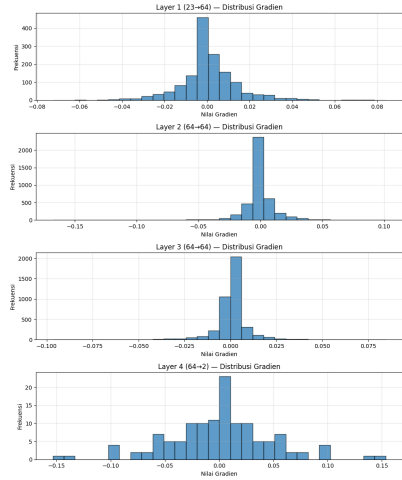
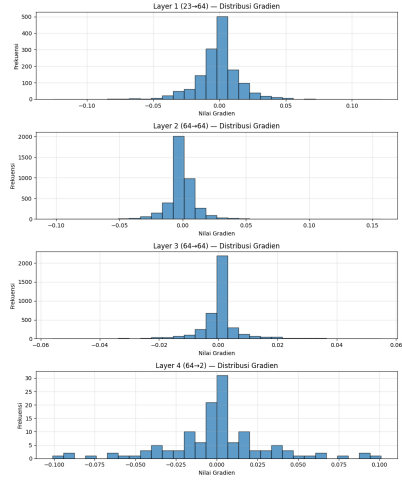
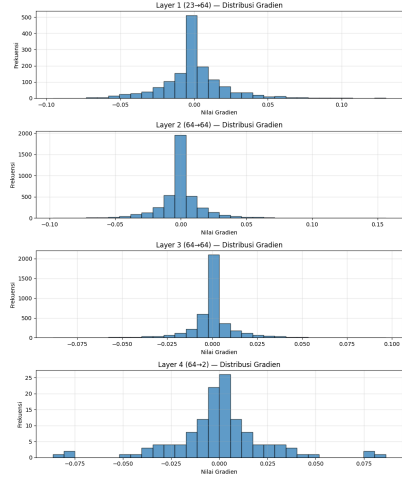
Gambar



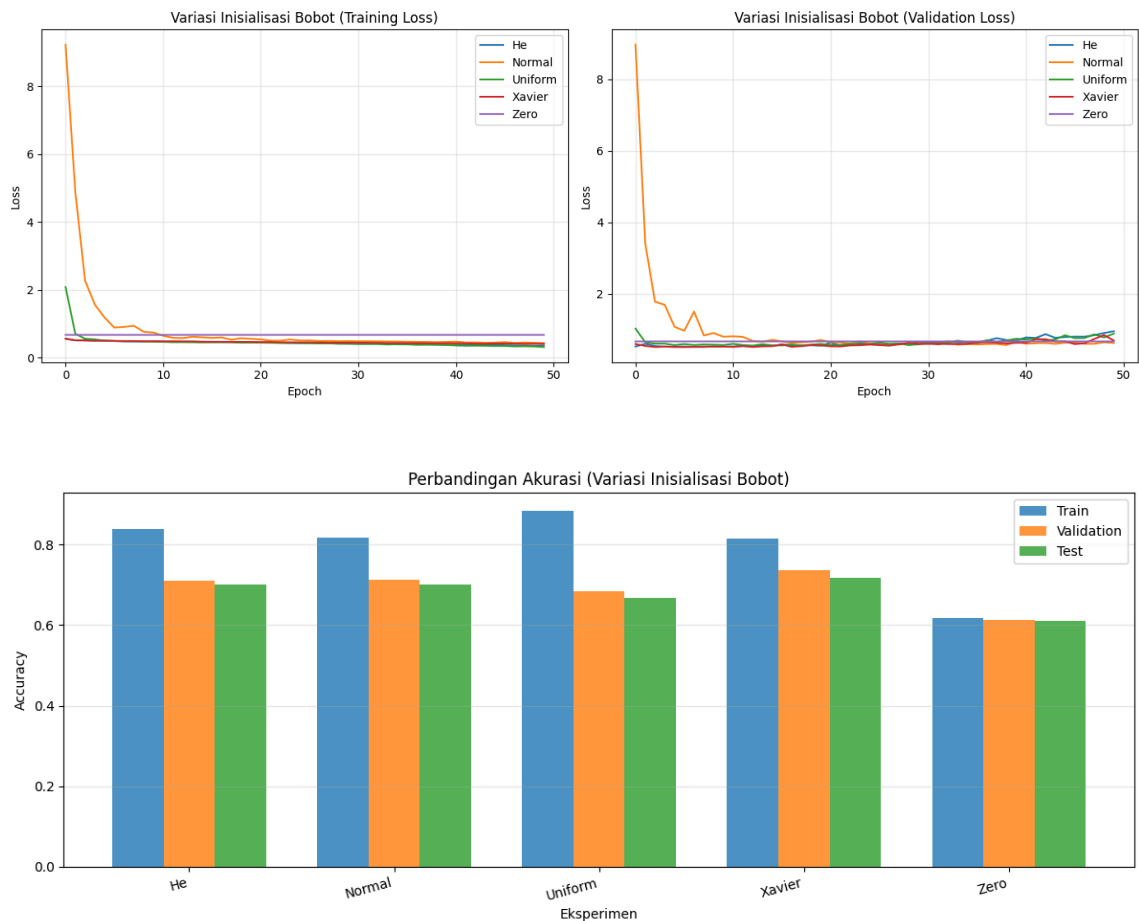
Distribusi Bobot

Metode	Gambar	Penjelasan
LR=0.001		Layer 1-3 berdistribusi normal di sekitar 0 dan hampir tidak bergeser dari inisialisasi awal akibat laju pembelajaran yang sangat rendah. Layer 4 (<i>output</i>) terdistribusi acak sebagai penyesuaian spesifik untuk memisahkan kelas target.
LR=0.01		Layer 1-3 berdistribusi normal proporsional di sekitar 0, menunjukkan pembaruan bobot yang paling optimal dan stabil. Layer 4 (<i>output</i>) terdistribusi asimetris sebagai penyesuaian akhir yang wajar untuk pemisahan kelas target.
LR=0.1		Layer 1-3 menunjukkan distribusi normal yang jauh lebih lebar dibanding variasi lainnya, menandakan pembaruan bobot yang sangat agresif. Layer 4 (<i>output</i>) terdistribusi sangat tersebar dan tidak beraturan, yang mengindikasikan model berisiko mengalami <i>overfitting</i> akibat langkah pembelajaran yang terlalu besar.

Distribusi Gradien

Metode	Gambar	Penjelasan
LR=0.001		Gradien seluruh <i>layer</i> terpusat sangat rapat di 0.00 dengan rentang sempit. Ini mengonfirmasi bahwa laju pembelajaran yang rendah mengakibatkan pembaruan bobot sangat minimal di setiap iterasi, sehingga model lambat mencapai titik optimal.
LR=0.01		Gradien Layer 1-3 berdistribusi normal di sekitar 0.00 dengan rentang stabil. Ini menunjukkan aliran gradien yang paling efisien untuk pembaruan bobot tanpa risiko ekstrem. Layer 4 memiliki persebaran terluas untuk optimasi klasifikasi akhir.
LR=0.1		Gradien menunjukkan rentang nilai terlebar di seluruh <i>layer</i>. Ini mengonfirmasi bahwa laju pembelajaran tinggi memicu pembaruan bobot yang sangat agresif dan fluktuatif, sehingga model berisiko tinggi mengalami <i>overfitting</i> atau sulit konvergen.

4. Pengaruh Inisiasi Bobot



1. Zero Initialization: Kegagalan Simetri

a. Analisis

Ini adalah metode dengan performa terburuk (Akurasi Test: 61.07%).

b. Alasan Teknis

Jika semua bobot dimulai dari nol, setiap neuron di layer yang sama akan menghitung nilai yang persis sama selama forward pass dan menerima gradien yang sama saat backprop. Akibatnya, neuron-neuron tersebut tidak bisa mempelajari fitur yang berbeda (masalah Symmetry Breaking). Model ini terjebak dan tidak berkembang optimal.

c. Grafik Loss

Terlihat garis ungu (Zero) pada grafik loss hampir datar dan berada di posisi yang jauh lebih tinggi dibanding metode lain.

2. Random Normal & Uniform

Normal (Akurasi Test: 70.07%) dan Uniform (Akurasi Test: 66.87%).

a. Analisis

Inisialisasi Normal menunjukkan lonjakan loss yang sangat tinggi di awal (mencapai >8) sebelum akhirnya turun. Ini menandakan bobot awal yang terlalu besar atau tidak terukur menyebabkan gradient instability di awal epoch.

b. Uniform

Menghasilkan akurasi training tertinggi (88.41%), namun akurasi test-nya rendah (66.87%). Ini adalah indikasi overfitting yang parah (gap $\sim 22\%$). Bobot awal yang terlalu luas jangkauannya membuat model menghafal data training terlalu cepat.

3. Xavier/Glorot Initialization

a. Analisis

Merupakan metode yang paling seimbang dalam eksperimen ini (Akurasi Test Tertinggi: 71.73%).

b. Alasan Teknis

Xavier dirancang untuk menjaga varians aktivasi dan gradien tetap stabil di setiap layer. Meskipun arsitektur Anda menggunakan ReLU (yang biasanya lebih cocok dengan He), Xavier menunjukkan generalisasi yang lebih baik pada dataset ini dengan gap train-test yang lebih sehat dibanding He.

c. Grafik Loss

Kurva merah (Xavier) adalah yang paling stabil dan halus sejak epoch pertama.

4. He/Kaiming Initialization

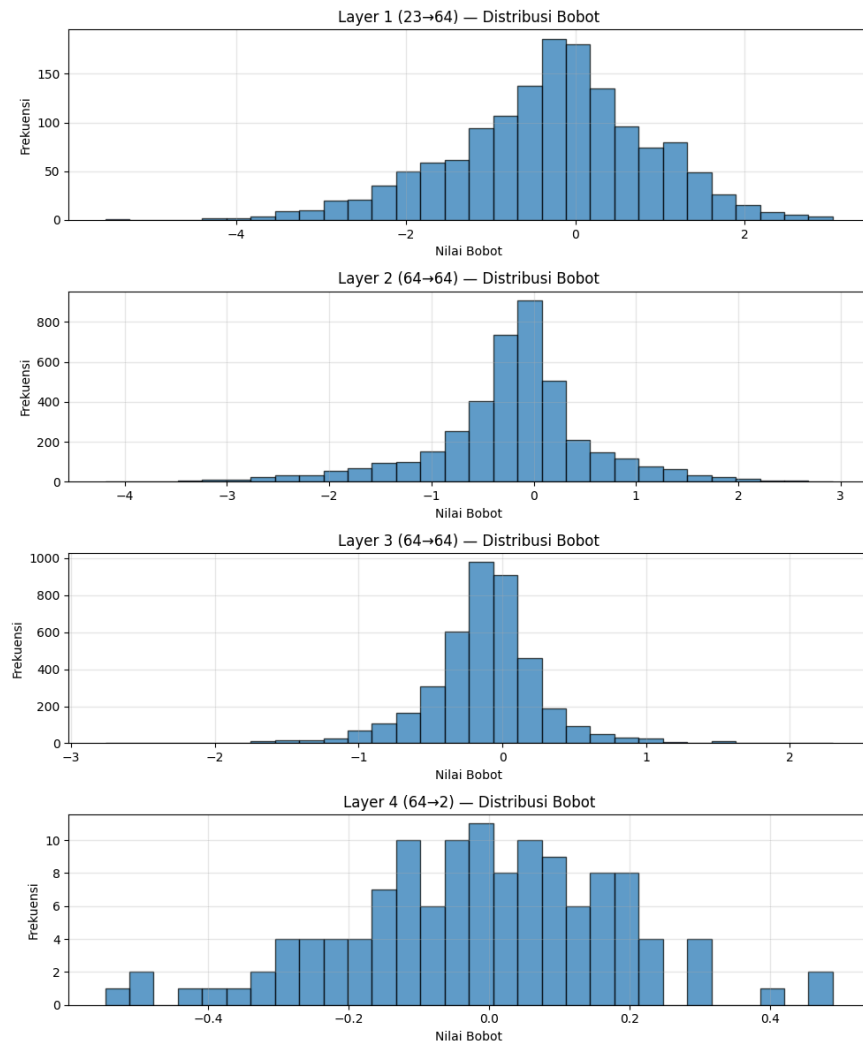
a. Analisis

Memberikan akurasi training yang kuat (83.99%) dan akurasi test yang solid (70.00%).

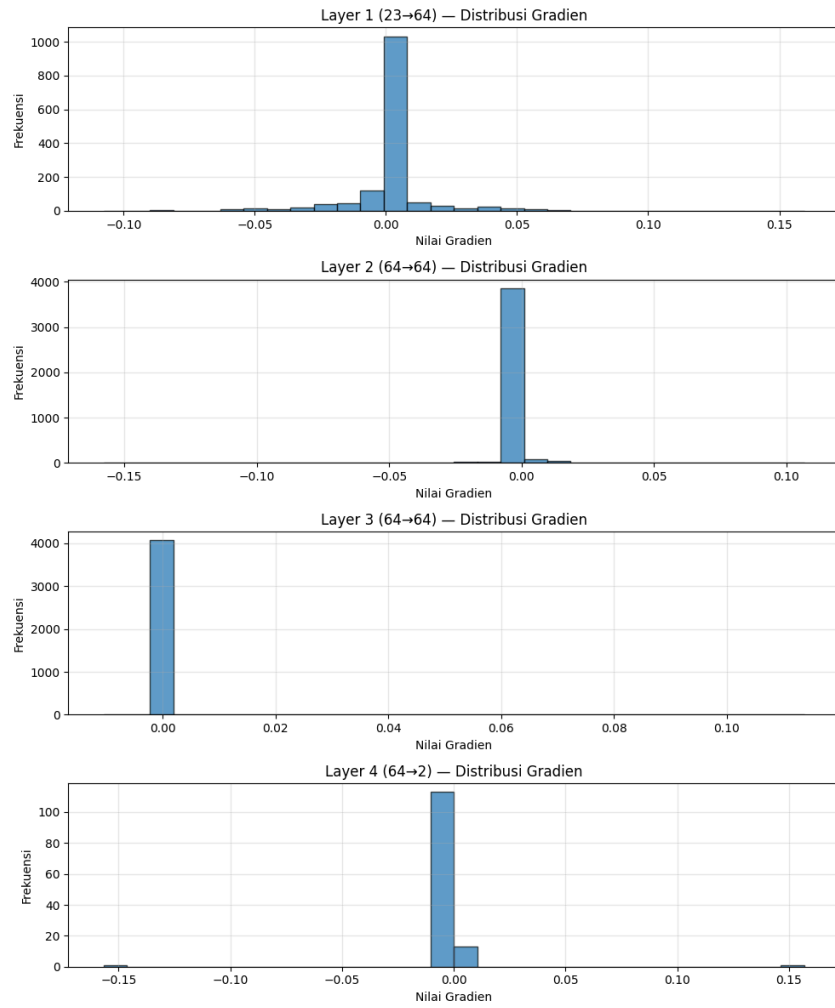
b. Alasan Teknis

Inisialisasi He dirancang khusus untuk fungsi aktivasi non-linear seperti ReLU. Ia mengompensasi fakta bahwa ReLU mematikan setengah dari unit inputnya (nilai negatif).

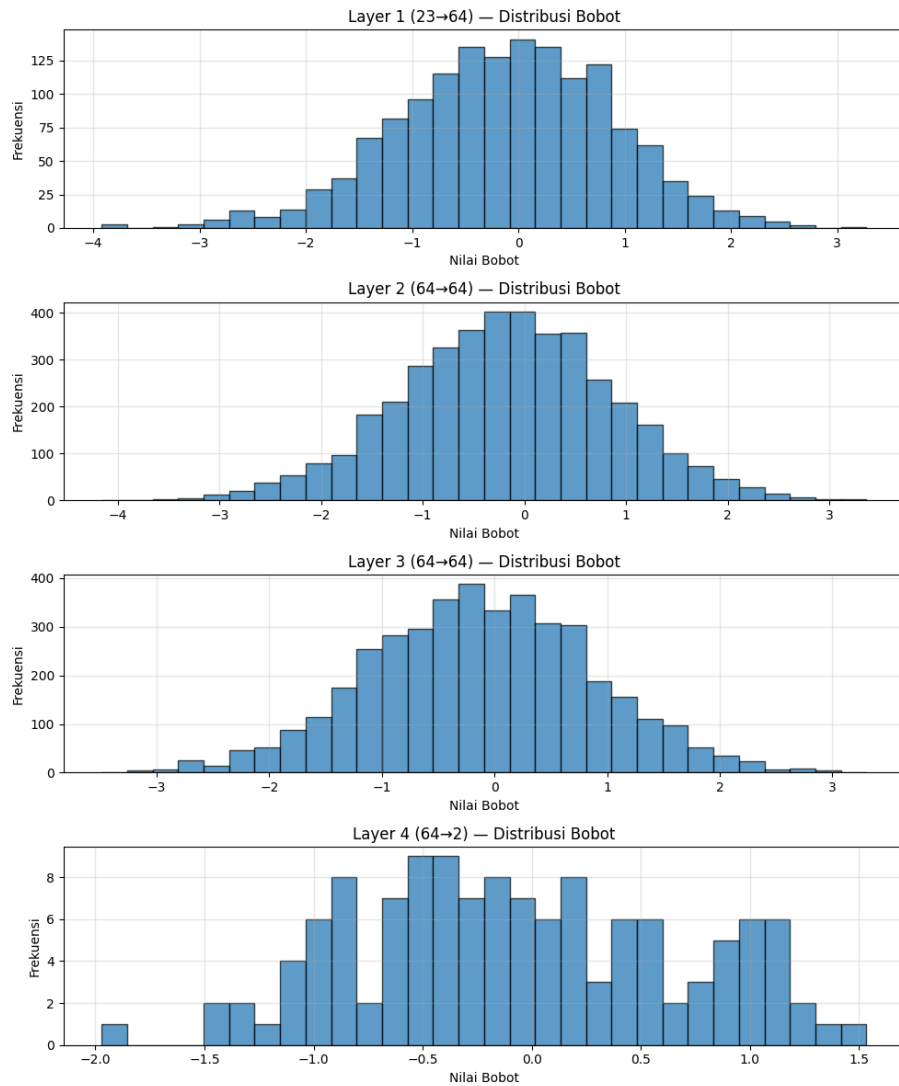
He konvergensi sangat cepat (lihat kurva biru yang langsung turun tajam di awal), namun pada kasus ini sedikit kalah generalisasi dibanding Xavier.



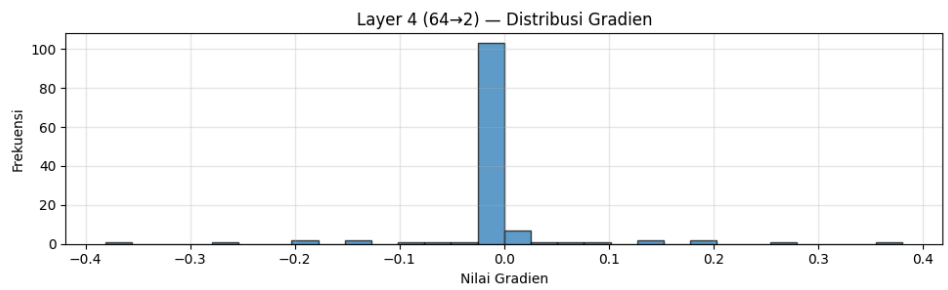
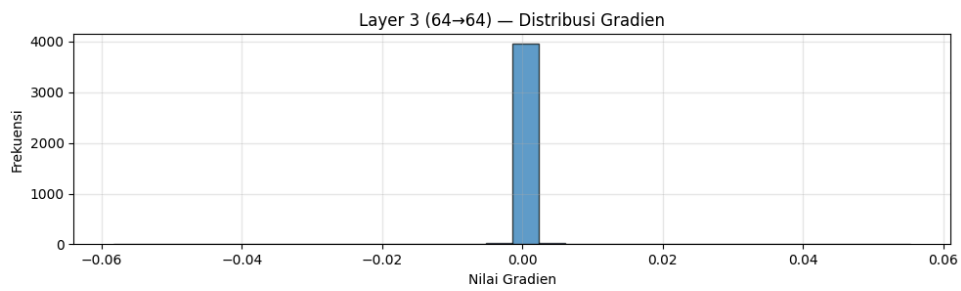
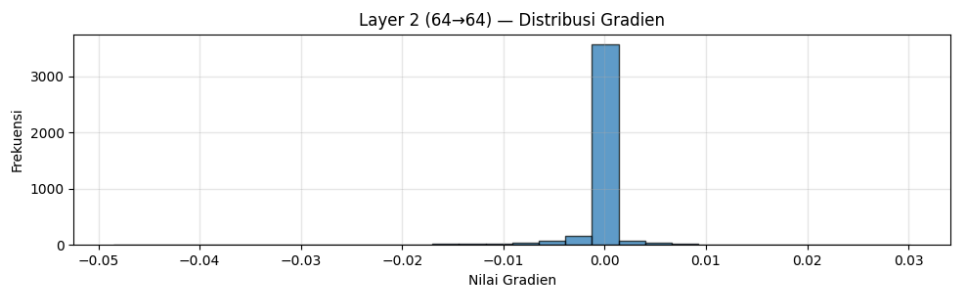
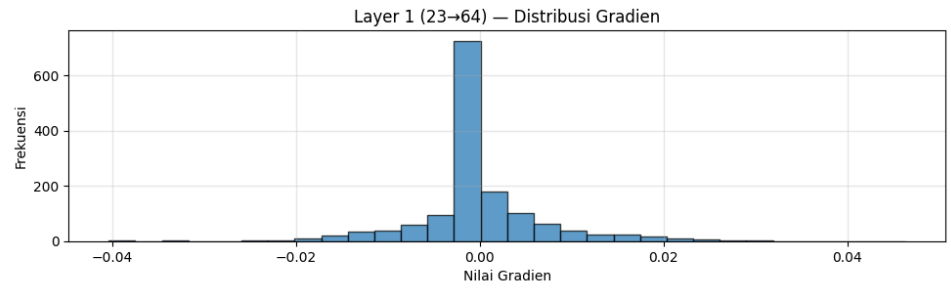
Inisialisasi: He



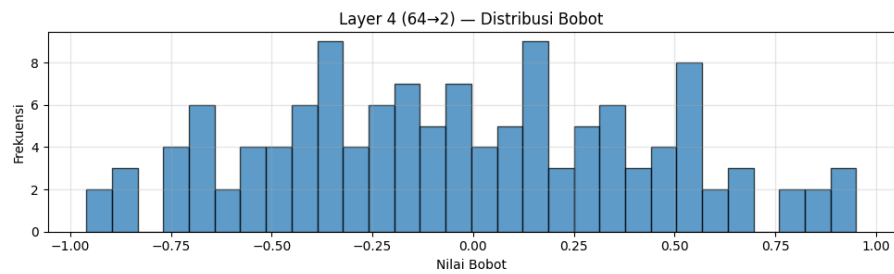
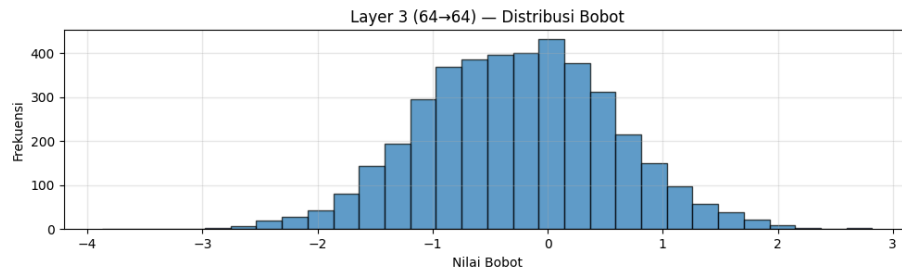
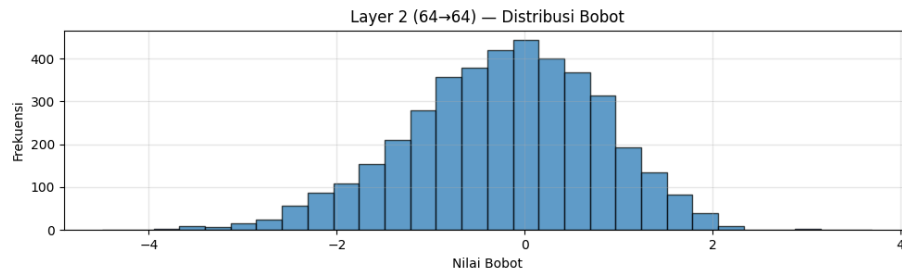
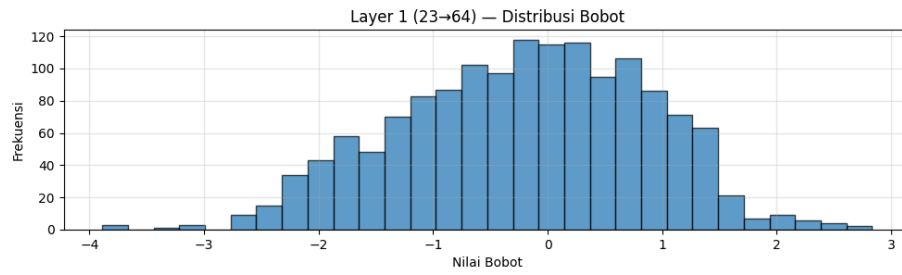
Inisialisasi: He



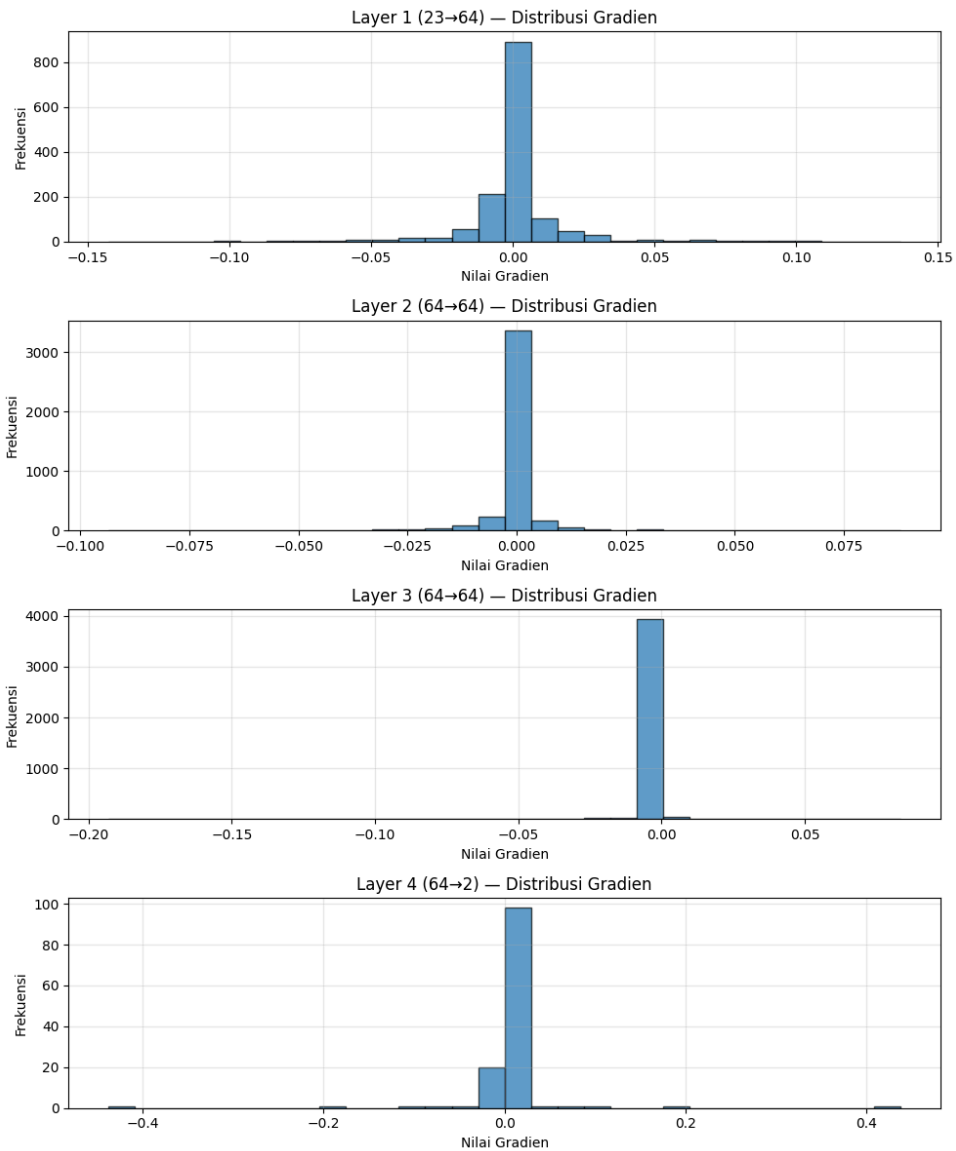
Inisialisasi: Normal



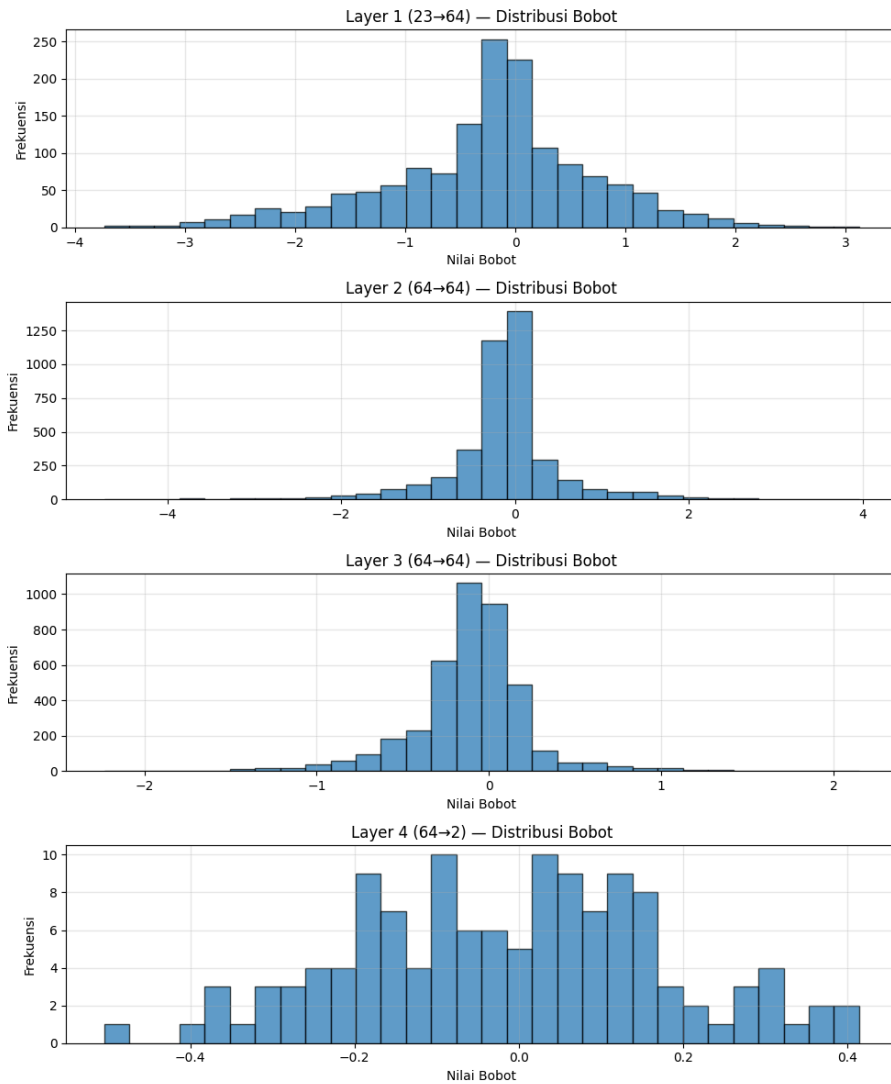
Inisialisasi Normal



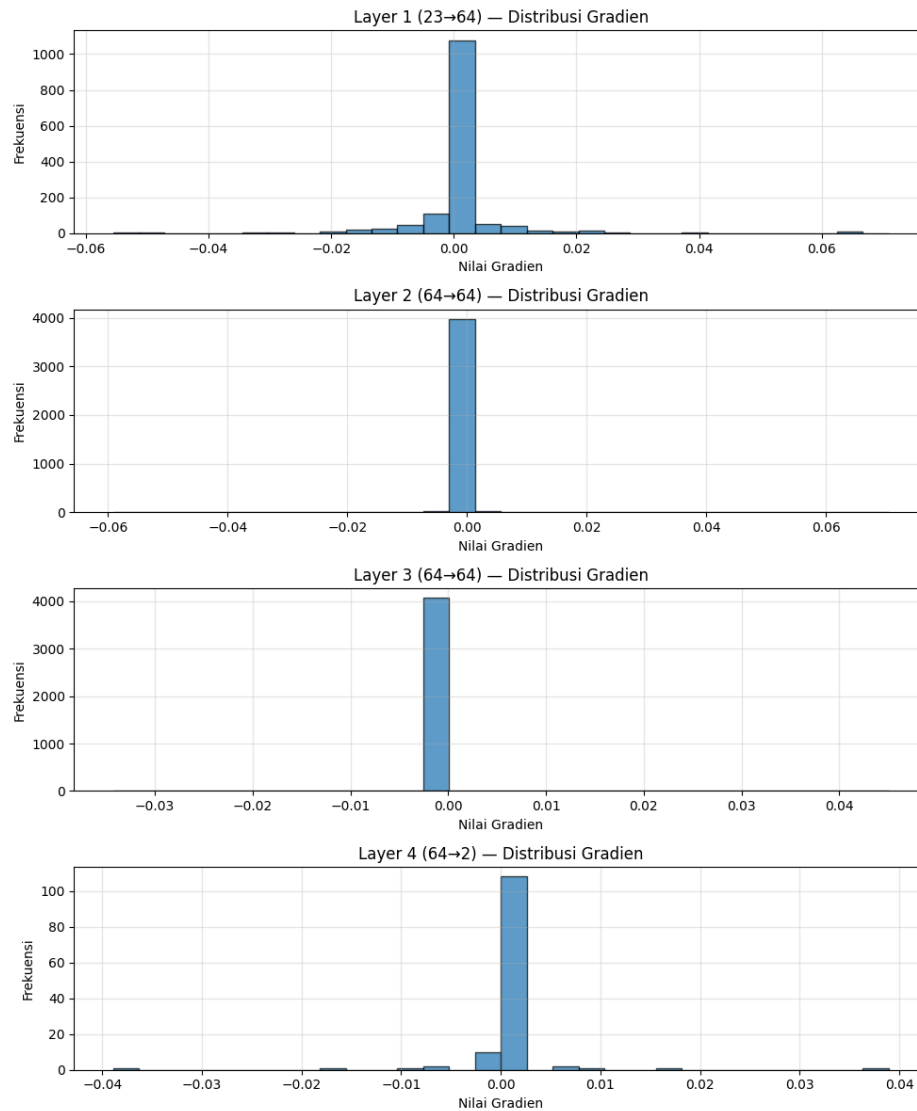
Inisialisasi Uniform



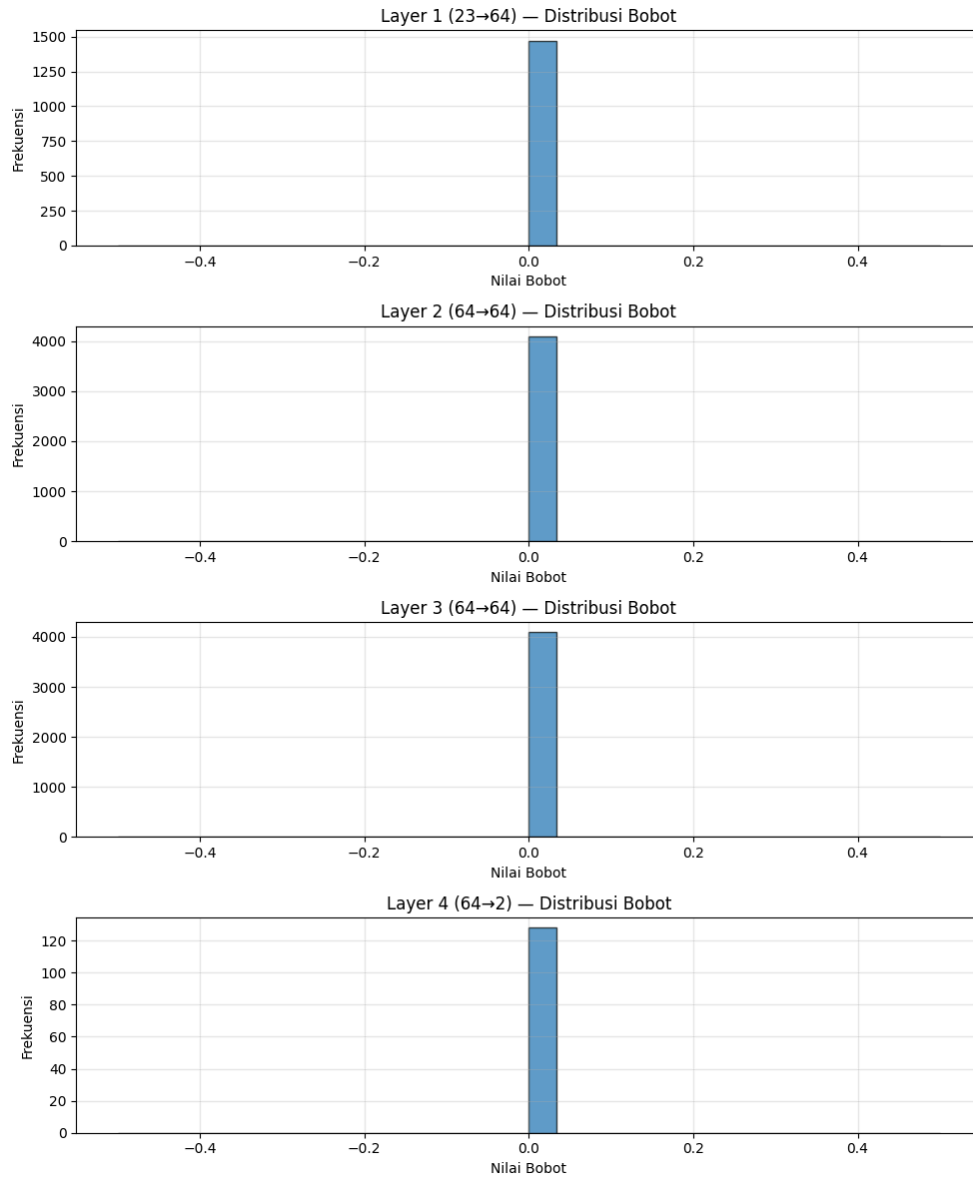
Inisialisasi Uniform



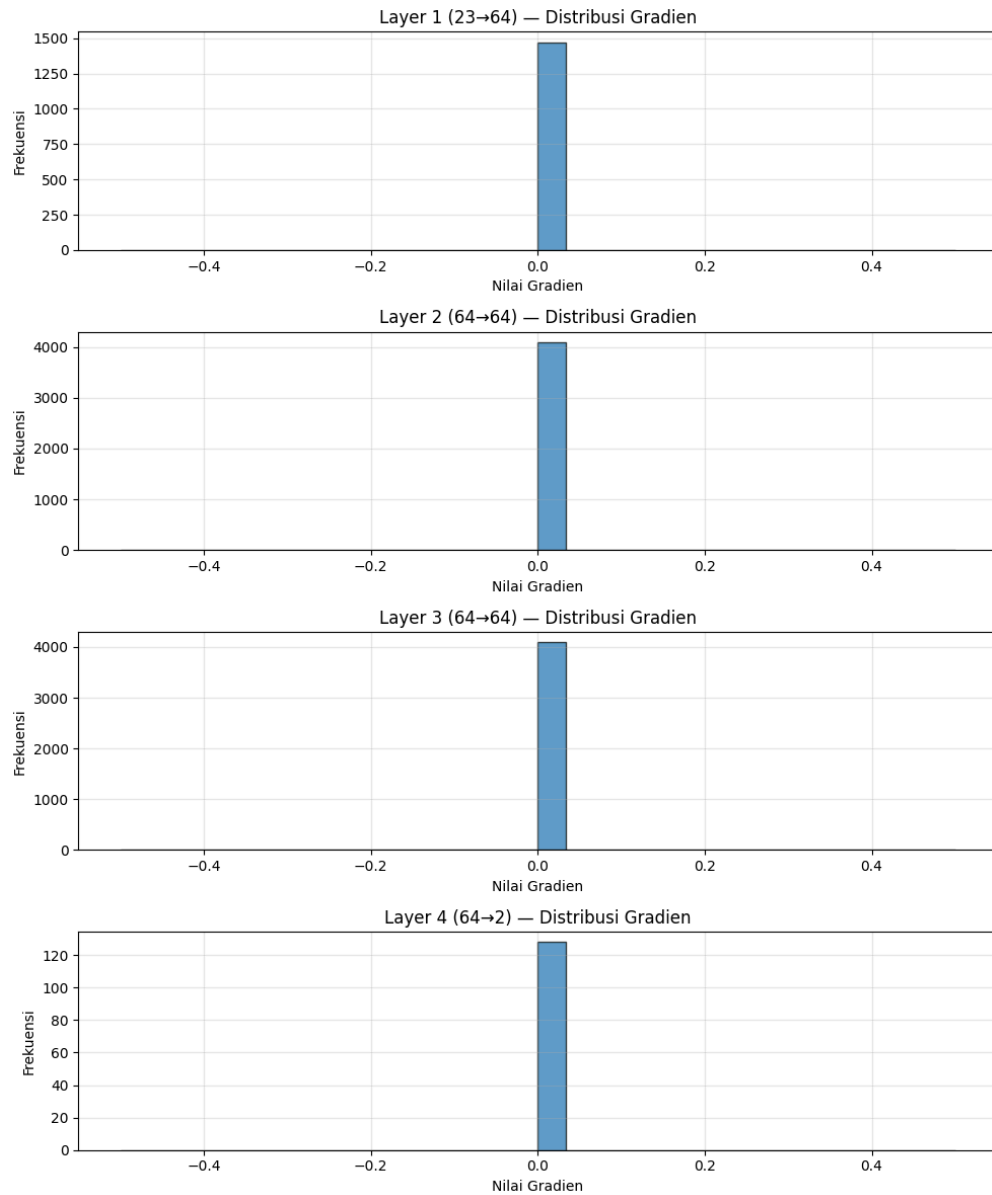
Inisialisasi Xavier



Inisialisasi Xavier



Inisialisasi Zero



Inisialisasi Zero

5. Pengaruh Regularisasi

Metode	Train Acc	Val Acc
Tanpa Reg	0.7874	0.7327
L1 ($\lambda=0.001$)	0.7713	0.7473
L2 ($\lambda=0.001$)	0.7839	0.7440

- a. L1 (Lasso): Menurunkan akurasi training secara signifikan (dari 78% ke 77%), namun menghasilkan gap terkecil antara Train dan Val. Ini menunjukkan L1 sangat efektif menekan kompleksitas model.
- b. L2 (Ridge): Memberikan keseimbangan terbaik. Meskipun akurasi trainingnya sedikit di bawah tanpa regularisasi, L2 menghasilkan Test Accuracy tertinggi (73.73%).

Kurva Loss

1. Training loss
Model tanpa regularisasi (biru) mencapai loss terendah paling cepat. Ini wajar karena model bebas melakukan "memorizing" pada data latih.
2. Validation loss
Kurva L1 (oranye) dan L2 (hijau) terlihat lebih halus dan berada di bawah kurva biru pada akhir epoch. Tanpa regularisasi, loss validasi mulai menunjukkan tren stagnan atau sedikit naik (gejala awal overfitting).

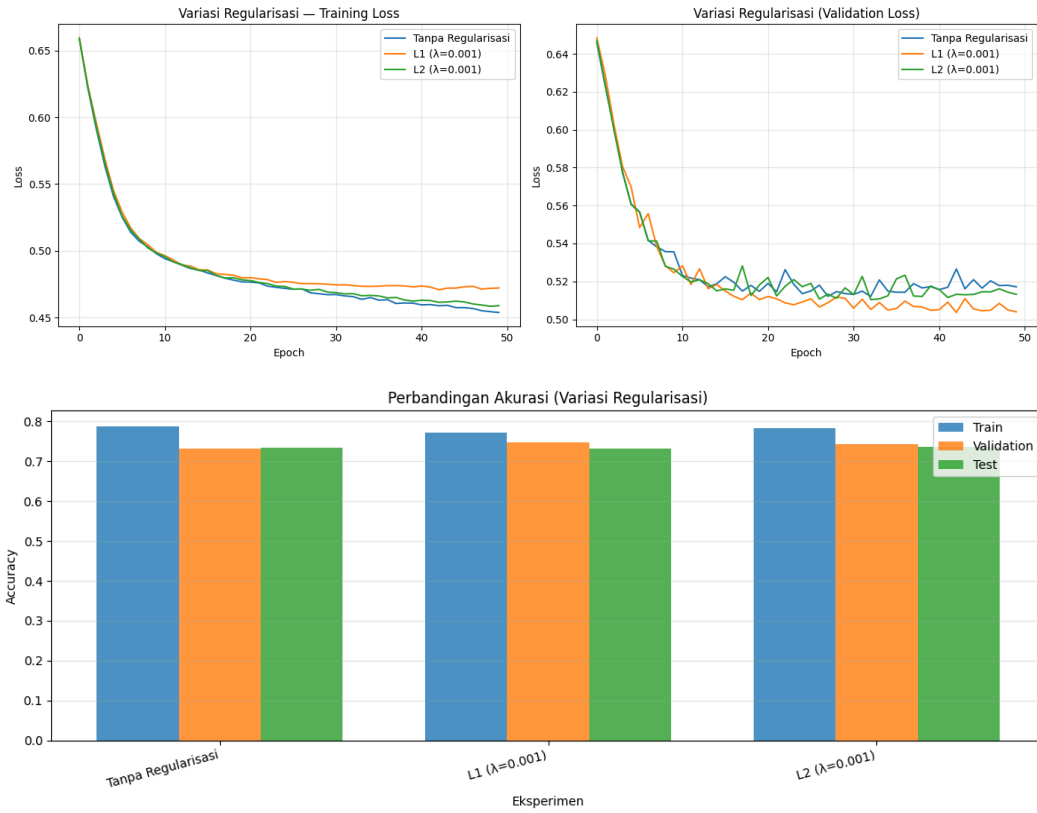
A) Weight Distribution

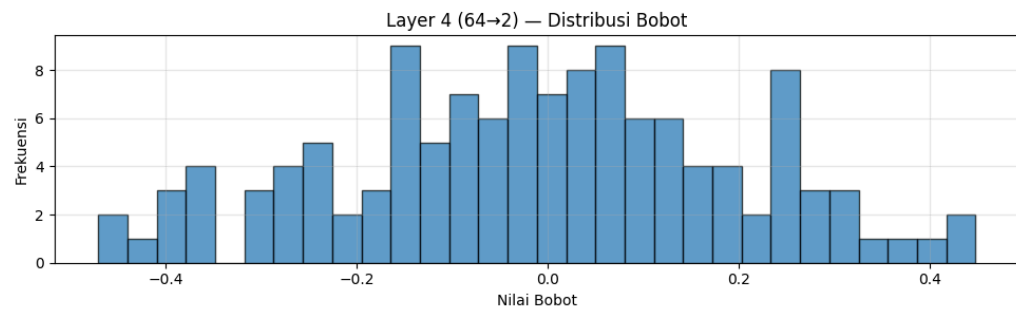
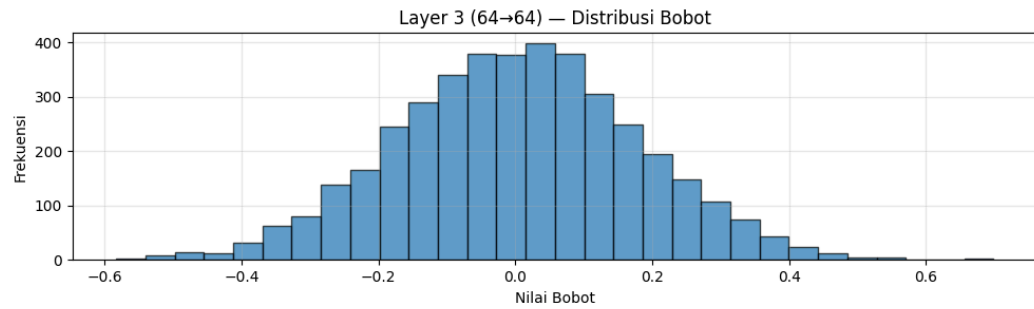
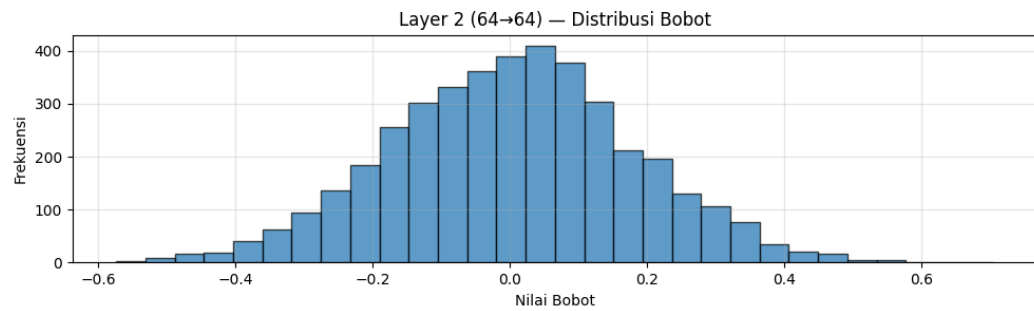
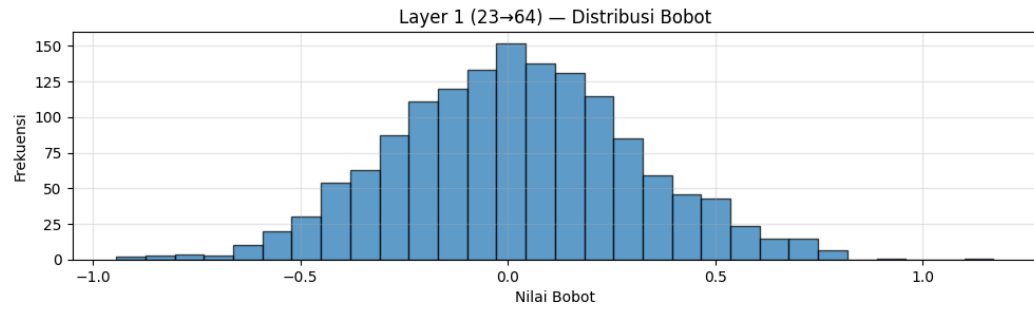
Layer 1 hingga 3 menunjukkan distribusi normal (Gaussian) yang cukup lebar, dengan rentang nilai antara -0.6 hingga 0.6. Sementara layer output menunjukkan persebaran yang lebih acak dan tidak beraturan. Tanpa penalti (L1/L2), bobot dibiarkan tumbuh bebas untuk menyesuaikan dengan fitur spesifik, yang menjelaskan mengapa akurasi training bisa lebih tinggi namun kurang general di data test.

B) Gradient Distribution

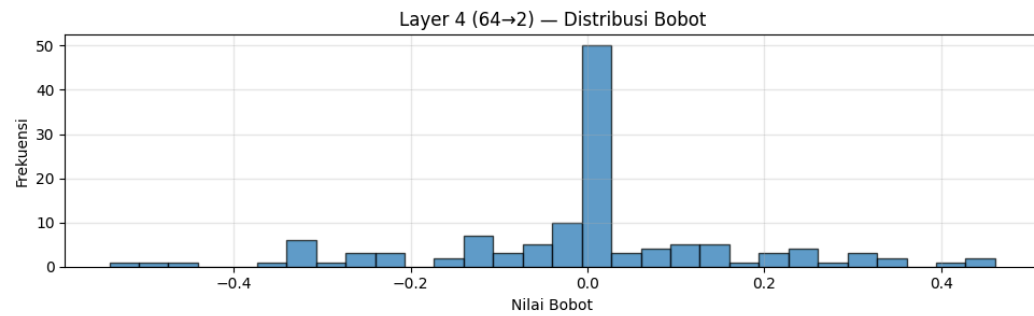
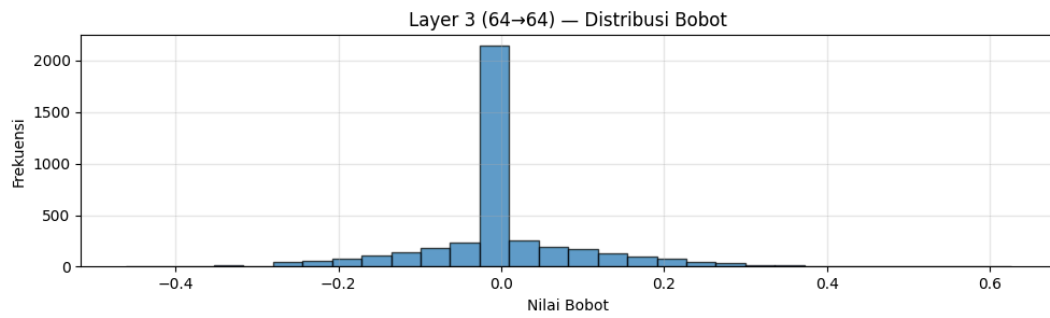
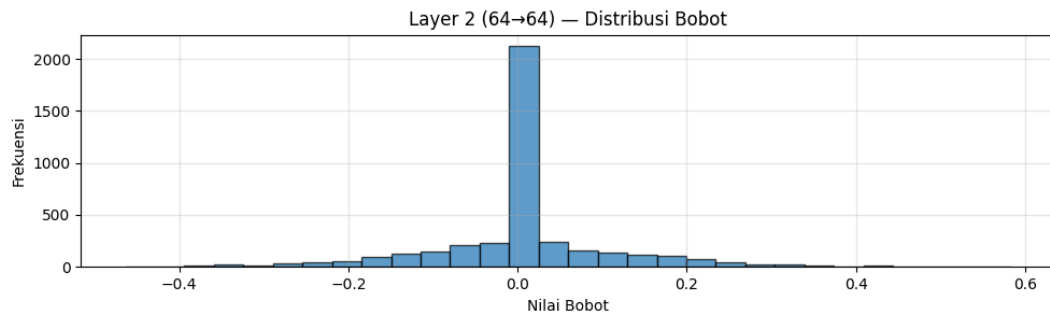
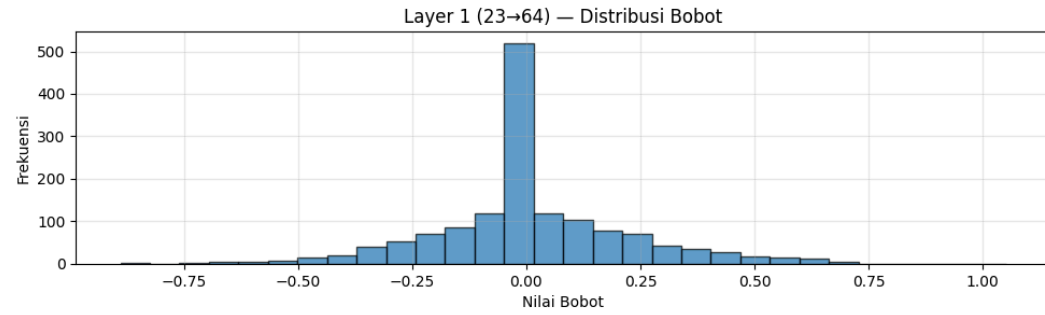
Mayoritas gradien terkonsentrasi sangat rapat di sekitar 0.000. Ini menandakan model sudah mencapai tahap konvergensi (stasioner) di mana perubahan bobot tidak lagi drastis.

Tidak terlihat adanya gradien yang meledak (semua berada di rentang kecil). Namun, konsentrasi yang terlalu rapat di nol pada layer awal (Layer 1) menunjukkan bahwa pembelajaran berjalan sangat lambat di akhir epoch karena sinyal error yang semakin mengecil saat mengalir mundur.

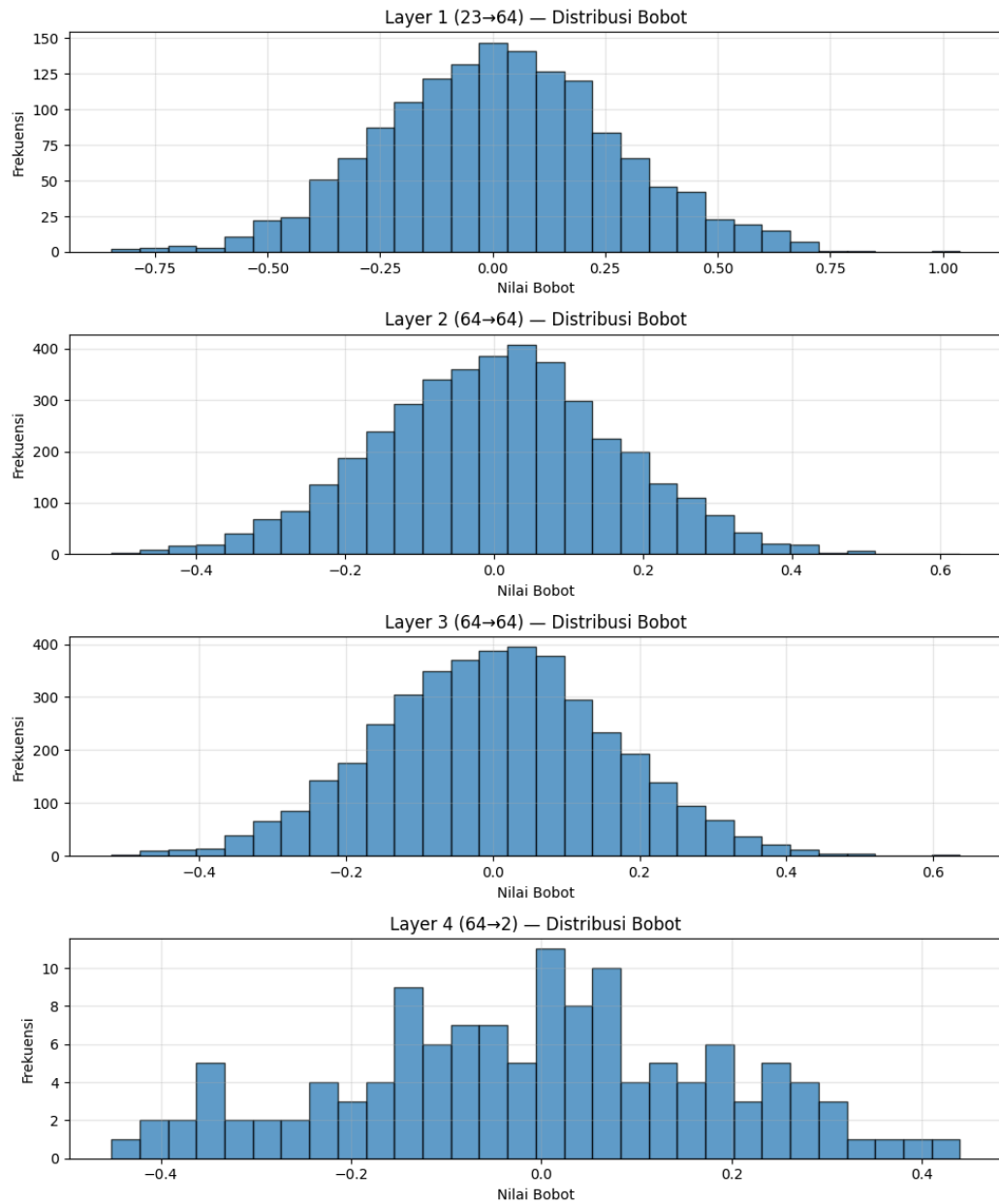




Tanpa Regularisasi



L1 ($\lambda=0.001$)



L2 ($\lambda=0.001$)

6. Pengaruh Normalisasi RMSNorm (Bonus)

Metode	Train Acc	Val Acc	Test Acc	Gap (Train-Val)
Tanpa Normalisasi	0.7803	0.7433	0.7273	3.70%

Dengan RMSNorm	0.8061	0.7347	0.7193	7.14%
----------------	--------	--------	--------	-------

Training Analisis

RMSNorm secara signifikan meningkatkan akurasi training (naik ke 80.61%). Ini membuktikan bahwa normalisasi membantu model untuk melakukan optimasi parameter dengan lebih agresif dan mencapai "fit" yang lebih dalam pada data latih.

Risiko Overfitting

Namun, peningkatan pada training tidak diikuti oleh data validasi/test. Gap antara Train dan Val meningkat hampir dua kali lipat (7.14%). Hal ini mengindikasikan bahwa pada arsitektur ini, RMSNorm membuat model terlalu cepat menyesuaikan diri dengan detail spesifik data training.

Analisis Kurva Loss

a. Training Loss

Kurva RMSNorm (oranye) turun lebih tajam di akhir epoch dibandingkan model standar.

b. Validation Loss

Terjadi anomali pada kurva oranye di grafik kanan. Terlihat lonjakan (spikes) yang sangat tajam dan fluktuatif setelah epoch ke-10.

Lonjakan ini biasanya terjadi karena RMSNorm menormalkan aktivasi berdasarkan akar rata-rata kuadrat (RMS), yang bisa menyebabkan perubahan gradien yang sangat besar jika statistik batch berubah-ubah, terutama jika tidak disertai dengan learning rate decay atau regularisasi yang kuat.

Mengapa RMSNorm berperilaku seperti ini?

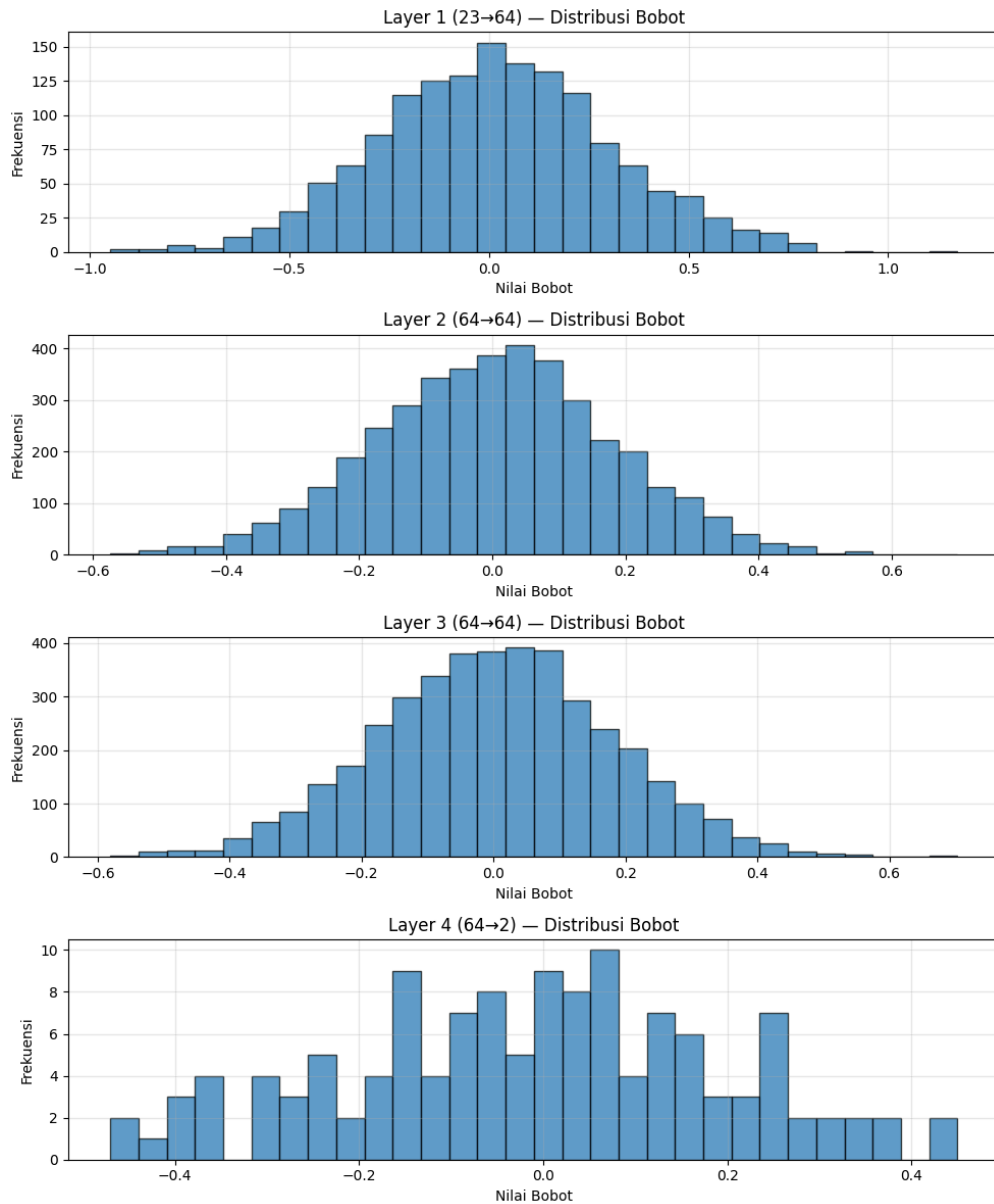
RMSNorm dirancang untuk menstabilkan distribusi input ke fungsi aktivasi dengan rumus

$$\hat{a} = \frac{a}{\sqrt{\text{mean}(a^2) + \epsilon}} \cdot \gamma$$

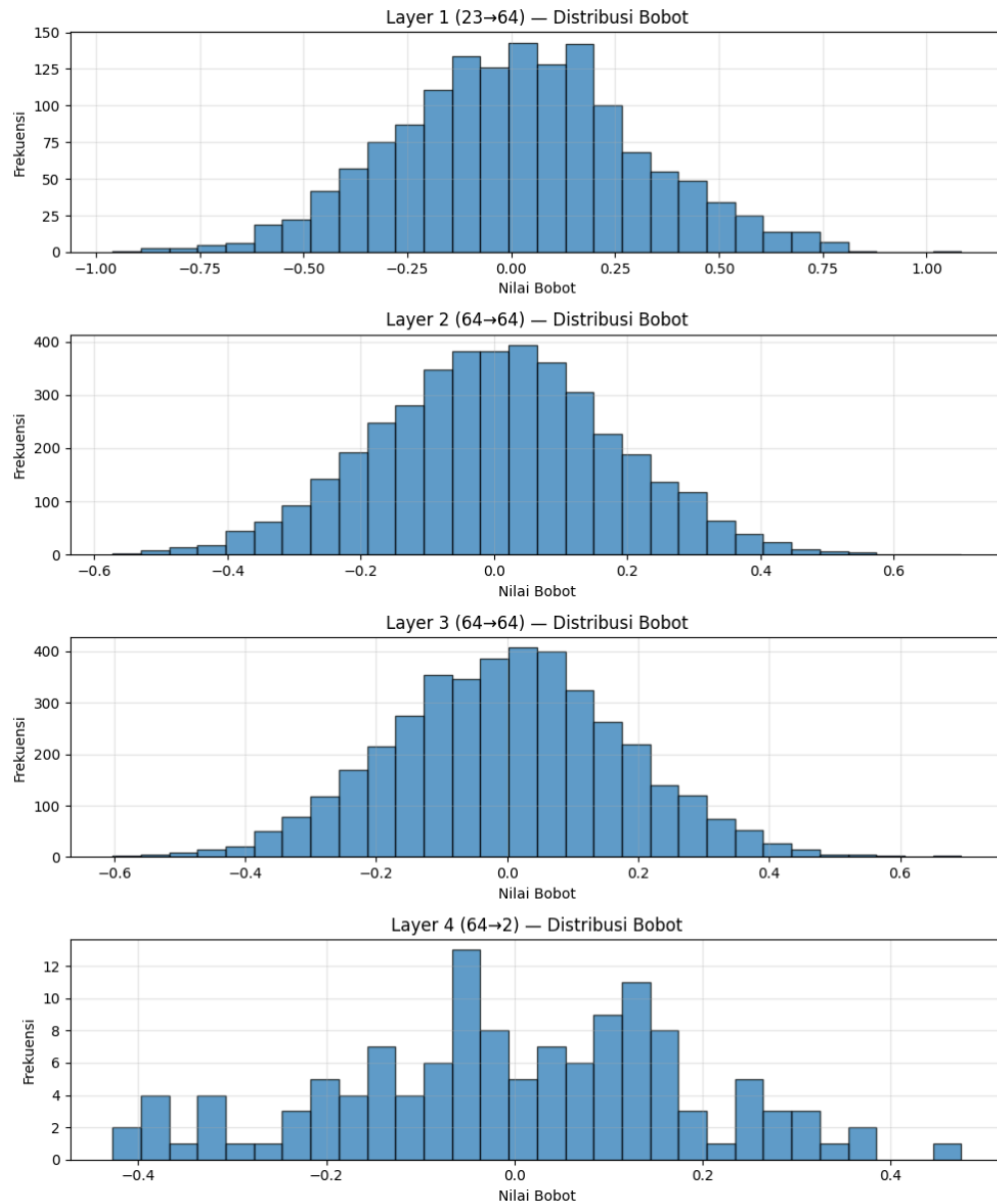
Dengan begini, RMSNorm menghilangkan ketergantungan pada skala input, sehingga gradien mengalir lebih konsisten. Inilah yang menyebabkan akurasi training melonjak cepat.

Tidak seperti Batch Normalization yang memiliki efek noise bawaan (yang membantu generalisasi), RMSNorm (mirip dengan LayerNorm) lebih

fokus pada stabilisasi sinyal internal. Tanpa regularisasi tambahan (seperti L2), model dengan RMSNorm menjadi sangat "haus" akan informasi data training sehingga melupakan generalisasi.



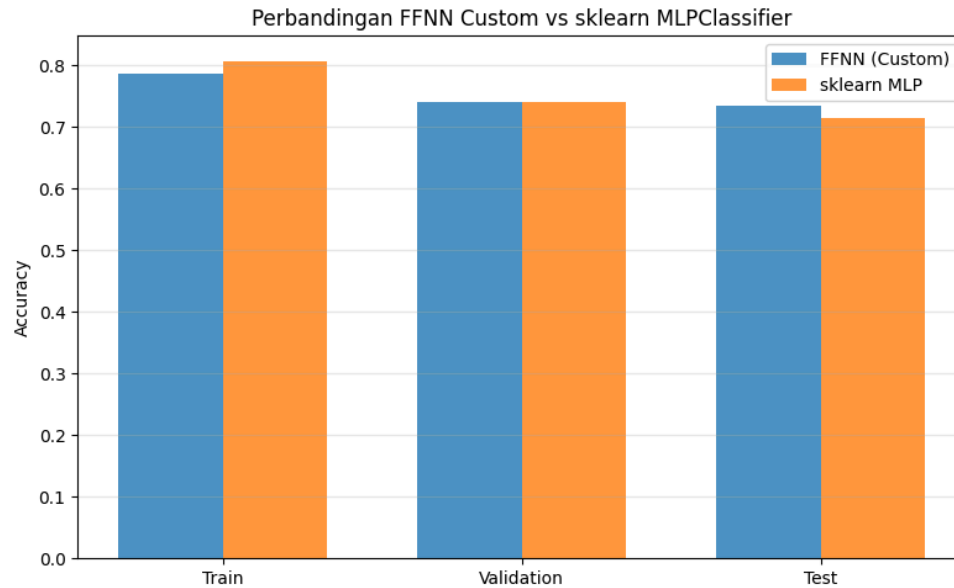
Tanpa Normalisasi



Dengan RMSNorm

7. Perbandingan dengan Library Sklearn

Metrik	FFNN (from scratch)	Sklearn MLP	Selisih
Train Accuracy	0.7871	0.8074	+2.03% (sklearn)
Val Accuracy	0.7400	0.7413	+0.13% (sklearn)
Test Accuracy	0.7353	0.7153	+2.00% (Custom)



Analisis Overfitting vs. Robustness

Model sklearn memiliki akurasi training yang lebih tinggi (80.74%), namun mengalami penurunan performa yang cukup signifikan saat bertemu data baru (Test Accuracy turun ke 71.53%). Gap antara Train dan Test mencapai 9.21%.

Model from scratch lebih konsisten. Meskipun akurasi trainingnya lebih rendah (78.71%), model ini hanya turun ke 73.53% pada data Test. Gap antara Train dan Test hanya 5.18%.

Jadi model custom memiliki kemampuan generalisasi lebih baik pada dataset ini dibandingkan implementasi default sklearn dengan parameter yang sama.

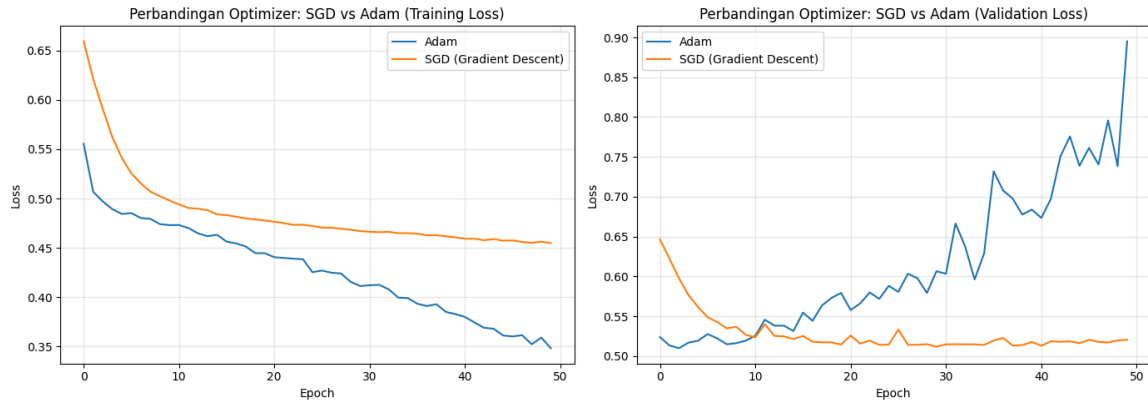
Kenapa hasilnya berbeda?

Meskipun hyperparameter (arsitektur, LR, batch size) disamakan, ada beberapa faktor teknis yang menyebabkan perbedaan.

1. Pada model from scratch menggunakan He Normal initialization secara eksplisit. Sklearn menggunakan varian Xavier/Glorot secara default. Perbedaan distribusi bobot awal ini sangat krusial bagi model ReLU untuk menghindari neuron mati (dying ReLU).
2. Jalur khusus Softmax + CCE yang dibuat (menggunakan rumus $\frac{y_{hat} - y}{N}$) seringkali lebih stabil secara matematis daripada library umum yang mungkin memisahkan kalkulasi keduanya demi fleksibilitas.
3. Sklearn mungkin menyertakan beberapa optimasi default (seperti momentum atau weight decay kecil) jika tidak dimatikan sepenuhnya,

namun hasil dari model yang dibuat menunjukkan bahwa implementasi SGD murni sudah cukup baik.

8. Perbandingan Kecepatan Konvergensi SGD dengan Adam Optimizer (Bonus)



Berdasarkan grafik perbandingan loss di atas, Adam optimizer menunjukkan kecepatan konvergensi yang lebih tinggi pada data training dengan training loss yang turun lebih cepat hingga mencapai 0.35, dibandingkan SGD yang cenderung stagnan di sekitar 0.46. Namun pada validation loss, SGD justru lebih unggul dengan loss yang stabil di 0.51, sementara Adam mengalami overfitting signifikan setelah epoch ke-10 dengan val loss yang terus meningkat hingga 0.90. Hal ini menunjukkan bahwa meskipun Adam lebih cepat konvergen pada data training, SGD menghasilkan model yang lebih general pada dataset ini.

IV. Kesimpulan dan Saran

A. Kesimpulan

Berdasarkan hasil pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan mengenai pengaruh berbagai hyperparameter dan konfigurasi terhadap performa model FFNN.

- Penambahan jumlah hidden layer (depth) meningkatkan akurasi training, namun model dengan depth lebih tinggi cenderung mengalami overfitting. Depth=1 terbukti memberikan generalisasi terbaik pada dataset ini.
- Peningkatan jumlah neuron (width) memberikan kapasitas representasional yang lebih besar, namun peningkatan akurasi validasi yang dihasilkan tidak sebanding dengan risiko *overfitting* yang meningkat.
- Pemilihan fungsi aktivasi berpengaruh signifikan terhadap performa model. Fungsi aktivasi Linear menghasilkan generalisasi terbaik, sementara ReLU paling agresif dalam mempelajari data training namun rentan terhadap fenomena *dying neuron* dan *overfitting*.
- Learning rate 0.01 terbukti paling optimal, karena nilai yang terlalu kecil memperlambat konvergensi sedangkan nilai yang terlalu besar memicu overfitting.
- Regularisasi L2 memberikan keseimbangan terbaik antara akurasi training dan generalisasi, sementara L1 paling efektif dalam mendorong bobot menuju nilai sparse.
- Adam optimizer menunjukkan kecepatan konvergensi yang lebih tinggi pada data training dibandingkan SGD, namun SGD terbukti lebih stabil dan menghasilkan model yang lebih general pada dataset ini.
- Model FFNN custom memiliki kemampuan generalisasi yang lebih baik dibandingkan sklearn MLP dengan gap Train-Test hanya 5.18% dibandingkan sklearn yang mencapai 9.21%.

B. Saran

Berdasarkan hasil pengujian ini, berikut beberapa saran yang dapat dipertimbangkan untuk pengembangan model selanjutnya:

- Mengimplementasikan *early stopping* untuk menghentikan training secara otomatis ketika *validation loss* mulai meningkat, sehingga dapat mencegah *overfitting* secara lebih efektif.
- Mencoba kombinasi regularisasi L1 dan L2 untuk mendapatkan manfaat dari kedua metode secara bersamaan.
- Menerapkan learning rate scheduler pada Adam, misalnya dengan menurunkan learning rate secara bertahap, untuk mengatasi masalah *overfitting* yang terjadi di epoch-epoch akhir.
- Melakukan pengujian nilai hyperparameter yang lebih granular di sekitar nilai optimal yang ditemukan, misalnya learning rate di antara 0.005 hingga 0.02, untuk menemukan konfigurasi yang benar-benar optimal.
- Selain akurasi, metrik evaluasi lain seperti precision, recall, F1-score, dan confusion matrix juga patut dipertimbangkan untuk mendapatkan gambaran performa model yang lebih komprehensif.

Dengan mempertimbangkan saran-saran di atas, diharapkan pengembangan model FFNN selanjutnya dapat mencapai performa yang lebih optimal.

V. Pembagian Tugas

No	Mahasiswa	NIM	Tugas
1.	Nazwan Siddqi Muttaqin	18223066	1. Mengerjakan loss function. 2. Mengerjakan implementasi <i>Dense layer</i>. 3. Mengerjakan penambahan turunan (<i>derivative</i>) untuk setiap <i>layer</i>.
2.	Surya Suharna	18223075	1. Mengerjakan <i>activation function</i>. 2. Mengerjakan dokumen laporan bagian pembahasan. 3. Mengerjakan dokumen laporan bagian hasil pengujian 5, pengujian 6, dan pengujian 7. 4. Mengerjakan <i>save & load mechanism</i>. 5. Mengerjakan bonus adam optimizer.
3.	Matthew Sebastian Kurniawan	18223096	1. Mengerjakan tahap inisialisasi (<i>initialization</i>). 2. Mengerjakan komponen layer. 3. Mengerjakan model Feed Forward Neural Network (FFNN). 4. Mengerjakan notebook untuk pengujian. 5. Mengerjakan bonus implementasi RMSNorm. 6. Mengerjakan dokumen laporan bagian hasil pengujian 1, pengujian 2, pengujian 3, dan

			pengujian 8.
--	--	--	--------------

VI. Referensi

Materi Kuliah IF3270 Pembelajaran Mesin pada bagian ANN: Perceptron dan ANN: FFNN - Sekolah Teknik Elektro dan Informatika Komputasi

Scikit-learn Documentation. Neural network models (supervised). scikit-learn.org

NumPy Documentation. Mathematical functions and array manipulation. numpy.org

Kaggle. Global Student Placement & Salary Dataset.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

VII. Lampiran

FORMAT DEKLARASI PENGGUNAAN AI SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Nazwan Siddqi Muttaqin/18223066, Surya Suharna/18223075, Matthew Sebastian Kurniawan/18223096

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 1 Feedforward Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Rendah. Untuk memastikan keselarasan teks satu dengan yang lainnya.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Medium. Digunakan sebagai teman diskusi dan menganalisis sesuatu.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18-03-2026



Nazwan Siddqi Muttaqin
18223066



Surya Suharna
18223075



Matthew Sebastian Kurniawan
18223096